

Resumable FFT Scheduling for Real-Time Spectral Analysis

Christian Kauten
Arhythmic Units
arhythmic.units@gmail.com

September 30, 2026
Technical report, manuscript version 3

Abstract

Spectral analysis can be inexpensive on average yet concentrate enough work in one audio callback to interfere with other processing. This report studies cooperative execution of a complete windowed-analysis pipeline on the engine thread. A dependency-ordered sequence covers packing, windowing, FFT butterflies, positive-bin reconstruction, smoothing, and output preparation. A balanced integer quota bounds scheduling credits per sample and publishes a complete spectrum at an exact hop; we derive input-retention, sample-age, and ownership requirements. A scalar and four-channel SIMD implementation is evaluated against matched scheduling controls and FFTW, PFFFT, and vDSP on an Apple M1 Pro. The confirmation campaign covers 1299 workloads with three process repeats in each of three prepared same-host sessions, including inverse jobs and complete overlap-save chains. For representative scalar unsmoothed analysis at $N = 4096$, $H = 1024$, 48 kHz, and 64-sample callbacks, the production analyzer’s simulated callback p99 is $3.417\ \mu\text{s}$ versus $9.500\ \mu\text{s}$ for immediate vDSP, while aggregate cost rises from 15.38 to 34.98 ns/sample and publication is delayed by 21.31 ms. Inverse and filtering controls retain bulk operations and do not inherit the analyzer’s bounded-operation contract. Results support a conditional cost–burst–age tradeoff, not a universal speedup or an audio deadline guarantee. The contribution is a reproducible implementation study within established time-distributed FFT research.

Keywords: fast Fourier transform; cooperative scheduling; real-time audio; spectral analysis; inverse transform; overlap-save filtering.

1 Introduction

Spectral analysis shares an audio engine’s processing budget even when it produces no audio output. An analyzer that consumes little CPU time on average can still concentrate window preparation, transformation, and display-data processing into a single host callback. Conversely, spreading those operations can add dispatch cost and delay the spectrum. For interactive audio software, the relevant objective is therefore a joint constraint on aggregate cost, processing bursts, and the age of the information delivered to the application.

This paper asks how to schedule complete windowed analysis on an existing engine thread while making those tradeoffs explicit. A batch FFT is an appropriate baseline when its call fits the available budget. A worker thread can move analysis away from that thread when asynchronous delivery is acceptable. Cooperative execution offers another choice: retain the state of an unfinished

analysis and advance it on successive engine sample calls. No additional thread is required, but the application must account for all stages and preserve the selected input until it is consumed.

Making the FFT resumable is insufficient on its own. A butterfly budget can leave frame-sized packing, reconstruction, and output passes at the boundaries. Furthermore, restarting as soon as a transform completes can change the frame interval even when the requested computation horizon is unchanged. These observations motivate a dependency-ordered sequence for the complete analyzer and a publication clock independent of early arithmetic completion. The approach builds on time-distributed transforms and cooperative processing chains; its contribution is the explicit contract for this windowed analyzer, not a new FFT factorization.

We specify the work-credit bound, exact publication cadence, input-retention requirement, cancellation behavior, and ownership of completed output. We also define comparisons that distinguish distributing a fixed implementation from choosing a different FFT or processing layout. This distinction matters because a lower callback tail can coexist with higher total cost, and an optimized batch implementation can outperform a finely scheduled one. The empirical question is which workloads justify the additional state and spectrum age, rather than whether suspension is universally preferable.

The implementation is the shared analyzer in the Fourier spectrum analyzer and Spectre spectrogram modules for VCV Rack. Fourier processes four independent channels in SIMD lanes; Spectre uses scalar samples. The contribution has three parts: an explicit work, cadence, retention, and ownership contract for complete analysis; a reproducible comparison that separates scheduling controls from optimized-library alternatives; and evidence locating both the benefits and costs of that design on one ARM64 host. The study also evaluates periodic inverse jobs and complete frequency-domain filtering controls, whose remaining bulk stages make the limits of butterfly-only scheduling observable.

Section 5 specifies the production analysis design, and Section 6 defines the inverse and filtering experiments. Sections 7 and 8 give the method and confirmation results. The historical campaigns remain separately identified in Section 9 and the appendices. The measured sources are archived explicitly; historical timings are not attributed to the current weighted scheduler. The results favor cooperative analysis when reducing concentrated callback work is worth additional aggregate cost and spectrum age, while preserving cases where optimized batch execution is preferable.

2 Related Work and Positioning

Time-distributed FFT execution has a substantial history. Lo and Lee construct butterfly work during acquisition and also combine scheduling with recursive reuse across shifted frames [5, 6, 7]. In convolution, Borß distributes partition work and staggers filter schedules to control combined load [8], while Hurchalla distributes transform work across incoming blocks on one thread [9]. Battenberg and Avižienis schedule forward transforms, spectral products, and inverse transforms across callbacks, using FFTW for leaf transforms and comparing cooperative with preemptive execution [10]. Thus both complete processing chains and optimized transform kernels have precedents within cooperative scheduling. Wefers treats manual preemption and transform subdivision more broadly [11]; suspension granularity remains an implementation choice rather than an inherent advantage of butterfly-level execution. Liu, Yan, and Zou explicitly study time-distributed FFT and inverse FFT execution for online control [12]. Neither transform suspension nor extending it to an inverse chain is a new principle of this report.

For spectral visualization, Průša and Holighaus use a worker thread and distinguish computation, polling, and repaint delay [13]. Their filter-bank representation differs from the fixed-window FFT

considered here, but the separation of execution context and visible-output latency is directly relevant. Sliding, hopping, and feedforward transforms offer another dimension of choice: reuse across overlapping windows [15, 18, 17]. Frequency-domain kernels can support tapered windows in such methods [19, 20]. Overlap reuse and time distribution can be combined; they are not mutually exclusive alternatives. Our implementation selects a complete window and starts a fresh transform for each frame.

Within this literature, the present contribution is an implementation contract and an empirical study: a complete windowed analyzer with an exact publication hop, retained input, cancellation, and ownership of published output, evaluated alongside periodic inverse jobs and complete transform–filter–inverse chains. The inverse and overlap-save chains are benchmark applications of the same scheduling principle; they do not introduce a new convolution method. We measure FFTW, Rack’s PFFFT wrapper, and Apple Accelerate/vDSP as practical optimized batch baselines [22, 23, 24]. Matched first-party and PFFFT hybrid controls distinguish work placement from changing the underlying FFT implementation. The comparison accounts for input frames, bins, window, normalization, output processing, numerical acceptance, and layout. It does not rank the unmeasured worker-thread or overlap-reuse alternatives. Section 7 defines the measurement boundaries; Appendix I gives the detailed literature comparison.

3 Signal and Execution Model

Let $x[n]$ be a real signal sampled at f_s Hz. A sample call advances one engine sample; a host callback can render D such samples. These are distinct measurement boundaries. A hop H specifies the interval between selected frame endpoints, not the host callback size. The transform length $N \geq 4$ is a power of two, and $H \geq 1$ is an integer. With continuously captured input and fixed hop, the frame endpoint is $t_r = rH$. The selected window is

$$u_r[m] = x[t_r - N + 1 + m]w[m], \quad 0 \leq m < N, \quad (1)$$

where w is an analysis window. Its unnormalized forward transform is

$$X_r[k] = \sum_{m=0}^{N-1} u_r[m]e^{-j2\pi km/N}. \quad (2)$$

The associated bin frequency is kf_s/N . Deferring computation changes when X_r becomes available; it does not change which samples enter this sum. Sample-index age measures the distance from a specified frame timestamp to publication, converted to seconds using f_s . It excludes wall-clock computation, device buffering, and display consumption. In particular, a transform computed in its endpoint’s sample call has zero endpoint-to-publication age but nonzero execution time.

For amplitude calibration, the finite-window coherent gain is $G_c = N^{-1} \sum_m w[m]$. An isolated interior bin-centered real sinusoid can be scaled using $2|X_r[k]|/(NG_c)$ when the window’s positive- and negative-frequency contributions do not overlap materially. DC and Nyquist do not receive the factor of two. This is amplitude calibration, not power-spectral-density normalization; the latter involves window energy. The plugin caches window samples and applies tabulated coherent-gain factors. Those factors should not be assumed to equal the exact finite-length sum for every window and length. The original FFT campaign uses explicit rectangular and periodic Hann samples and does not certify display calibration. The pipeline campaign instead uses a coherent-gain-normalized periodic Hann window.

Real-input packing uses $M = N/2$ complex samples; the analyzer retains $K = M + 1$ nonnegative-frequency bins, including DC and Nyquist. The standard radix-2 traversal requires $B = B_r(N) =$

$(N/4)\log_2(N/2)$ butterflies. Appendix A preserves the butterfly equations, arithmetic-order proof, resumable state transition, and real-spectrum reconstruction.

4 Why Schedule the Complete Pipeline?

The original real FFT suspends a conventional ordered butterfly traversal. With B butterflies and requested horizon H , it executes $q = \lceil B/H \rceil$ butterflies per sample call and completes after $L = \lceil B/q \rceil \leq H$ calls. The original modules immediately restart completed transforms, so their steady-state frame interval is L , which need not equal H . For $N = 4096, H = 2048, L = 1878$: the frame rate is about 9.1% faster than requested, and temporal smoothing based on the nominal hop decays too quickly. Appendix B retains the completion proof, cadence table, timestamp diagram, and smoothing derivation.

Budgeting butterflies also leaves substantial boundary work. The original path packs and permutes a frame in bulk, reconstructs all real-spectrum bins on the completing call, and processes magnitude/display arrays in separate passes; some of those paths allocate. For $H = \rho N$ with fixed $\rho > 0$, the butterfly quota grows as $O(\log N)$, while preparation and reconstruction still impose $O(N)$ boundary passes. Appendix C gives the cost model and ownership analysis. These two limitations motivate an exact frame clock and a work sequence covering the complete analysis, not just its FFT.

5 Complete Analysis over One Hop

The analyzer maintains one ordered work sequence and one publication boundary per frame. It schedules windowing and packing, butterflies, positive-bin reconstruction, magnitude processing, and per-bin output on the engine thread. Every stage preserves the dependencies of uninterrupted evaluation; pausing changes the placement of work, not the selected signal frame. Unused conjugate bins are omitted. The shared `SpectrumAnalysis<T>` implementation supports scalar samples and independent SIMD channels with the same scheduling contract.

5.1 Dependency order and work bound

Let $M = N/2$, $K = M + 1$, and $B = (M/2)\log_2 M$. Table 1 defines the operations in each stage. Let $p = 4$ while rebuilding window coefficients and $p = 1$ for a clean window cache. Let o be the compile-time output weight: one by default, or two for Fourier’s four-channel coordinate mapping. The ordered scheduling sequence consumes W credits, where

$$W = pM + B + (1 + o)K = \frac{N}{4}\log_2(N/2) + \frac{(p + 1 + o)N}{2} + 1 + o. \quad (3)$$

All packing completes before any butterfly, all butterflies before any reconstruction, and the complete prefix sum before any range smoothing. A sample quota may cross stage boundaries. The dependency graph is unchanged when execution pauses; it therefore admits the same induction argument as the butterfly traversal in Appendix A. Arithmetic-order preservation applies to pauses within this implementation; it does not imply bitwise identity to other FFT kernels or compiler configurations.

For call $s \in \{1, \dots, H\}$, assign the quota

$$q_s = \lfloor sW/H \rfloor - \lfloor (s - 1)W/H \rfloor. \quad (4)$$

The cumulative credit count after s calls is $\lfloor sW/H \rfloor$. Credits are exhausted and the frame is published on call H , with at most $\lceil W/H \rceil$ credits per call. Weighted operations execute at their

Credits	Stage	Scheduled work
pM	Pack	Each pair occupies p credits: read two retained samples, prepare/apply window weights and store the permuted pair at the first credit; skip the remaining credit.
B	Transform	Execute one complex butterfly and advance traversal indices.
K	Reconstruct	Recover one bin, take its magnitude, and append a prefix sum if frequency smoothing is enabled.
oK	Output	Each bin occupies o credits: apply enabled smoothing and invoke the output callback at the first credit; skip the remaining credit.

Table 1: Production scheduling credits. Dirty window preparation has weight four; Fourier’s coordinate output has weight two. Other operations have weight one. Credits account for ordered work, not equal-duration operations.

first credit; later credits can be bookkeeping only, so arithmetic may finish before publication. For $H = \rho N$ with fixed $\rho > 0$, this is $O(\log N)$. A quotient/remainder accumulator implements the rule without per-sample division. The quota is an elementary balanced schedule applied to the complete dependency-ordered analysis. Appendix D preserves the dispatch listing, proof, and clean/dirty-cache examples.

For fixed W, H , any D consecutive active sample calls consume at most $\lceil DW/H \rceil$ credits, including intervals that cross frame boundaries. This follows by telescoping the quota differences and relates sample-level accounting to a host block. It applies per analyzer, without configuration changes, reset, or pauses; it is not a duration bound.

The per-call bound includes scheduled window/cache rebuilds and bounded module output callbacks. Construction and host lifecycle callbacks are excluded; constant input/control work and an $O(\log N)$ plan-index calculation remain. Decomposing every stage into bounded operations removes $O(N)$ boundary passes from steady-state engine analysis; the quota distributes those operations. Reweighting changes placement but does not equalize operation costs, constrain an arbitrary user callback, or bound operating-system interruptions.

5.2 Retained samples, plans, and publication time

Let the frame endpoint be $t_r = rH$, counting continuously captured samples from zero. Frame selection occurs on call t_r , and the last scheduling credit and publication occur on call $t_r + H - 1$. The last actual output may precede its final credit; it remains private until publication. The nominal sample-index ages at publication are therefore

$$A_{\text{newest}} = (H - 1)/f_s, \quad A_{\text{mid}} = ((N - 1)/2 + H - 1)/f_s. \quad (5)$$

These ages do not include elapsed computation or display delay. Successive publications are exactly H samples apart. This convention differs from the original next-call publication in Equation (17). Startup uses zero padding; a full first window is not awaited. With changing hops the endpoints satisfy $t_{r+1} = t_r + H_r$, and each frame uses its latched H_r .

Incremental packing must retain the selected samples while capture continues. A preallocated ring of capacity $C = N_{\text{max}} + H_{\text{max}}$ suffices for the prepared maximum window and hop; no whole-frame capture copy is needed. The lifetime proof and maximum-size table reuse are retained in Appendices D and E.

5.3 Smoothing, configuration, and output ownership

Reconstruction obtains magnitudes and, when frequency smoothing is enabled, builds a prefix sum before any range average is emitted. Output applies an inclusive fractional-octave magnitude mean, optional temporal averaging, and the module’s display conversion. These are magnitude means, not power means or constant-Q analysis. The window normalization and display units are preserved. Appendix D records the equations, control conventions, cache behavior, and sample-level hop quantization.

Settings latch at each frame start; reset and sample-rate changes cancel partial analysis. Window and smoothing caches rebuild inside scheduled units, and length changes logically clear input and averaging history. Storage is prepared outside the sample path; increasing the supported hop in a sample-rate callback can allocate. Freeze behavior differs between the two modules, so continuous-input timestamp formulas do not apply across pauses.

Each output bin is written to a producer-owned destination. Only a complete frame is published through a single-producer/single-consumer triple-slot mailbox using acquire/release atomic exchange. The consumer retains its slot until its next successful exchange, and the producer never overwrites that slot. A slow consumer may miss intermediate snapshots. Fourier uses one curve mailbox; Spectre uses one mailbox per history column, with image history and recoloring on the UI thread. Lifecycle work, other shared controls, UI work, and host scheduling remain outside the quota; the contract is not a broad claim of hard real-time safety.

6 Inverse Jobs and Complete Filtering Chains

Forward analysis is only one use of a resumable transform. The comparison campaign also evaluates independently released inverse transforms and complete frequency-domain filtering chains. These are benchmark applications of the reusable FFT classes; neither visualization module produces audio or invokes these paths. Their schedules retain bulk buffering operations, so the bounded operation contract of Section 5 applies only to the complete analysis implementation. This distinction lets the measurements expose the remaining concentration of work in the inverse and filtering controls.

6.1 Normalized inverse jobs

An inverse job accepts all N complex bins in natural order and produces all N complex time samples,

$$z[m] = \frac{1}{N} \sum_{k=0}^{N-1} Z[k] e^{j2\pi km/N} = \frac{1}{N} \overline{\text{FFT}(\bar{Z})[m]}. \quad (6)$$

The spectrum need not be Hermitian. No window is applied or inverted, and the factor $1/N$ belongs to every adapter’s measured work. In `OnTheFlyIFFT<T>`, buffering copies and permutes the input and conjugates the retained coefficients. Each subsequent step executes one forward butterfly or conjugates and normalizes one output. Thus the transform has $B_c(N) + N$ resumable steps, with $B_c(N) = (N/2) \log_2 N$. Buffering remains a timed $O(N)$ pass outside that step count.

The streaming job fixture releases a new spectrum on sample $t_r = rH$, independently of any forward analyzer. After the transform, the harness copies all N results to its canonical destination, giving

$$W_{\text{inv}} = B_c(N) + 2N \quad (7)$$

scheduled units in the first-party control. The immediate variant executes them on release; the balanced variant uses Equation (4) with $W = W_{\text{inv}}$. Both pay the same bulk input preparation

at release. Their release-to-publication delays are respectively $d = 0$ and $d = H - 1$ samples. A native-library job executes its transform, required conversion, normalization, and canonical stores immediately. These jobs publish N complex outputs every H calls; they do not define an audio stream with $N = H$, perform overlap-add, or inherit the analyzer's spectrum-center age.

6.2 Overlap-save processing and delivery

The complete-chain fixture follows conventional FFT convolution principles [11]. Let $v[n]$ be the common complex input stream, zero before startup, and let $g[\ell]$ be a causal FIR of length L . At endpoint t_r , the selected frame is $u_r[m] = v[t_r - N + 1 + m]$. With $G[k]$ the N -point transform of the zero-padded filter, the chain computes

$$c_r = \text{IFFT}(\text{FFT}(u_r) G), \quad b_r[j] = c_r[N - H + j], \quad 0 \leq j < H. \quad (8)$$

The product is binwise, and both transforms are full complex transforms. The last H samples agree with linear convolution when $N - H \geq L - 1$: each retained index then has every required input within the selected frame, so none of its filter terms wraps circularly. Discarding the first $N - H$ outputs is overlap-save extraction. No analysis or synthesis window is used.

The two fixtures are identity and $g = [1/2, -1/4, 1/8]$. Both require $H \leq N - 2$ and execute the same path, including all N spectral multiplications; the identity does not bypass the transforms. The three-tap filter is a correctness and application fixture, not a claim that FFT convolution is economical for such a short filter. This experiment evaluates neither a windowed STFT reconstruction nor partitioned long-filter convolution.

The first-party chain schedules forward butterflies, N multiplications, one inverse-buffer operation, inverse butterflies and normalization, and H output stores. Its accounting is therefore

$$W_{\text{chain}} = 2B_c(N) + 2N + H + 1. \quad (9)$$

The inverse-buffer operation charged as one unit contains $O(N)$ work. At frame release, an additional $O(N)$ ring-to-frame copy and forward buffer/permutation pass execute outside the quota. Consequently, $\lceil W_{\text{chain}}/H \rceil$ bounds scheduled units per call, but does not bound their primitive operations by the analysis scheduler's formula. The native-library chains execute the same functional pipeline immediately, including layout conversion and normalized inverse output. All chains include continuous delivery and a common dependent checksum that keeps every delivered sample observable to the optimizer.

Publication exposes a block at $p_r = t_r + d$. Playback starts in that call and delivers $b_r[j]$ at $p_r + j$. Since this sample corresponds to convolution output index $t_r - H + 1 + j$, the input-to-delivery delay is

$$D_{\text{chain}} = H - 1 + d = \begin{cases} H - 1, & \text{immediate,} \\ 2H - 2, & \text{balanced,} \end{cases} \quad (10)$$

in samples, or D_{chain}/f_s seconds. This is the implementation's buffering delay relative to the causal convolution output; any filter phase delay is separate. Successive blocks join without gaps after startup because each contains H outputs and publications are H samples apart. Device buffering and callback-end visibility add delay beyond this sample index model.

6.3 Independent output checks

Alternating sparse non-Hermitian spectra have known inverse sequences of the form $a + be^{j2\pi 7m/N} + ce^{-j2\pi 3m/N}$, allowing every inverse output to be checked without reusing a tested transform. For

the chains, direct time-domain identity/FIR evaluation checks every delivered sample and every published block, including startup zeros and retained-input wraparound. These synthesis checks require $|a - r| \leq \epsilon \max(1, |r|)$ for each complex output, with $\epsilon = 2 \times 10^{-5}$ for float and 10^{-10} for double; non-finite values fail. This pointwise absolute-or-relative budget differs from the processed-spectrum norm policy used for analysis. Correct fixture output and exact logical delivery do not by themselves demonstrate audio-device deadline compliance.

7 Experimental Method

The confirmation study evaluates work placement, practical implementation cost, and numerical correctness separately. Its unit of comparison is a fully specified workload: operation, precision, transform length, hop, callback size and origin, independent channels and instances, settings, state, load, and cache policy. Transform, analysis, inverse-job, and filtering boundaries are never pooled into a backend ranking.

7.1 Implementations and included work

Table 2 distinguishes scheduling controls from comparisons between implementations. The native hybrid retains an indivisible PFFFT call while scheduling its surrounding preparation and output stages. The scheduled batch control executes that same task sequence at the frame endpoint; the balanced hybrid distributes it over the hop. Comparing either with the production analyzer also changes transform layout, fusion, plan capacity, storage, and band-index arithmetic. It measures a practical tradeoff, not an isolated cost of suspension.

Scalar analysis uses common float input bytes containing DC, two tones, and deterministic noise; double adapters promote those same bytes. Matched adapters select the same frame endpoints and produce $K = N/2 + 1$ magnitudes. Stationary settings use coherent-gain-normalized periodic Hann windows, either without smoothing or with one-third-octave averaging and temporal coefficient $\alpha = 0.8$. Live-cache cases alternate Hann/Blackman–Harris windows and frequency-smoothing settings. Startup, independent four-channel cases, and explicit parameter transitions retain separate identities.

Window coefficients and control parameters are stored in float even for double analysis; input/window multiplication and transform arithmetic use the selected scalar precision. The reference consumes these same rounded windowed samples, so numerical acceptance does not certify the window generator or display calibration. Core and legacy smoothing compute interval endpoints with binary32 arithmetic, while the native analysis and hybrid adapters use binary64 arithmetic before flooring to bin indices. A boundary can therefore select a different bin despite identical

Comparison	Shared work	Interpretation
Legacy batch / incremental	Same real FFT, frame copy, full reconstruction, and output processing	Placement of the legacy FFT’s work.
PFFFT scheduled batch / hybrid	Same arithmetic, task dispatch, plans, caches, and retained input	Placement of native FFT and surrounding work.
Production core / native analysis	Matched input frames and required outputs; different implementations	Practical cost, burst, storage, and age tradeoff.

Table 2: Comparison roles. Legacy controls use fixed frame cadence rather than the original modules’ immediate restart. Native analysis includes PFFFT, FFTW, and vDSP where supported.

nominal octave width. Each is checked against its declared interval convention. Cross-family comparisons match the analysis task rather than asserting bitwise-identical smoothing outputs; the scheduled-batch/hybrid pair shares its exact convention.

Analysis timing includes capture, retained input, windowing, transformation, required output conversion, smoothing, and output stores. Inverse-job timing includes input buffering, normalized inverse evaluation, and all output stores. The chain includes retention, forward transformation, spectral multiplication, inverse transformation, overlap-save extraction, and sample delivery (Section 6). Fixture generation, construction, planning, and warmup are excluded. Startup omits warmup but still excludes construction. Isolated transforms include their adapter’s required buffering and conversions; they are not advertised as bare provider-kernel calls. The comparison campaign contains no running Rack audio engine or display consumer.

7.2 Host, compilation, and preparation

All measurements used one Apple M1 Pro laptop (10 CPU cores, 16 GiB memory), macOS 26.6.2 ARM64, and Apple Clang 21.0.0. The first-party driver used `-O3`, `-funsafe-math-optimizations`, and `-march=armv8-a+fp+simd`; numerical acceptance applies to that executable, not to every compiler transformation or architecture. The measured first-party source is revision `12ac3236e332` plus the recorded runner-only metadata fix; its exact archive is identified in Appendix K. No production DSP changed during the campaign.

External adapters use Rack’s PFFFT revision `74d7261be17c`, FFTW 3.3.10 with NEON support, and Apple Accelerate/vDSP [23, 22, 24]. FFTW uses fresh single-threaded MEASURE plans with no imported wisdom; per-process plan text and build provenance are retained. Plan construction is measured separately from execution. Library-specific native scratch and plan storage are not fully captured by the C++ allocation audit. Missing native allocation counts are unknown, not zero. vDSP is tied to the recorded system framework; these observations do not compare operating systems.

The host was on AC power with Low Power Mode disabled. Each of three confirmation sessions followed a separately documented preparation and at least 180 seconds without benchmarks, builds, tests, or reporting. Groups ran serially in rotated order, with deterministic randomized workload order within groups. These sessions are temporal blocks on the same laptop and night; they are not independent machines or days. Ordinary macOS services and Codex remained possible confounds. No core affinity, real-time priority, frequency isolation, or continuous temperature trace was available. Pre/post warning snapshots do not establish constant temperature.

7.3 Workload coverage and repetition

Two pilots, comprising 3945 successful processes, preceded frozen confirmation. A failed preliminary launch was rejected by the provenance gate before any measurement and is retained separately. The confirmation design partitions 1299 distinct nominal workloads, counting timing passes separately, into five groups, with three fresh process repeats per workload in each session: 11691 successful confirmation processes. Only 16 exactly duplicate transition configurations shared by the pilot plans were removed; no contender or unfavorable case was dropped on timing grounds.

The primary grid spans $N \in \{2048, 4096, 16384\}$ and callback sizes $D \in \{16, 64, 256\}$ for streaming cases, at 48 kHz. Extensions investigate channel count, supported double precision, frame phase, background DSP, sample rate, startup/live-cache state, cache disturbance, short hops, single-sample callbacks, and configuration transitions. The retained frozen manifests enumerate exact combinations; this is a structured workload set, not the Cartesian product of every factor.

Group	Workloads	Measured hops	Callback observations/process
Primary	408	2048	8192 / 32768 / 131072
Standard extensions	719	512	2056 / 8192
Short hop	110	4096	2368
Single-sample callback	46	64	65536
Transitions	16	64	37 / 1024

Table 3: Frozen confirmation groups. Callback counts depend on each group’s hop and block size; they are observations, not independent repeats. Isolated transforms use 1024 frames instead of the streaming hop count. Stationary streaming cases use $\lceil N/H \rceil + 64$ warmup hops plus the requested phase offset; transition cases start without warmup.

Unsupported precision/provider combinations are absent rather than silently replaced.

7.4 Timing, age, and aggregation

A continuous throughput pass estimates aggregate cost; a separate pass records each synchronous simulated callback. A monotonic clock measures elapsed wall time, including preemption. Empty timer intervals are retained without subtraction, and observation storage and CSV serialization remain outside timing. Streaming cost is nanoseconds per engine sample for all configured instances and included background DSP; isolated cost is nanoseconds per transform. Callback p99 and observed maximum are reported in microseconds. A callback of D samples has nominal interval D/f_s seconds. Exceeding it is a simulated compute-budget exceedance, not a measured audio underrun. Sub-tick observations cannot resolve individual operation costs. Sparse FFT bursts can escape p99, and a maximum is neither a stable tail quantile nor a worst-case execution-time bound [30].

An untimed replay checks actual publication indices against each adapter’s contract. Spectrum-center age adds $(N - 1)/2$ samples to endpoint-to-publication delay. The production analyzer adds $H - 1$ samples; immediate batch analysis adds zero. Callback-end visibility adds the remainder of the block. Inverse jobs use release-to-publication delay; chains use input-to-delivery delay. These quantities exclude repaint delay and smoothing response; replay does not measure timing-dependent consumer contention.

Costs average processes within sessions, then weight the three session means equally. Callback p99 uses nearest-rank process quantiles, medians within sessions, and the median of session medians. Tables retain observed session ranges and maxima where indicated. These ranges are descriptive: callbacks are not pooled as independent replicates, and no inferential confidence interval, tail convergence, or precise rare-event probability is claimed. Three prepared sessions do not remove setup, planning, or layout bias [29, 28].

7.5 Numerical coverage and evidence checks

Scalar analysis replay checks every published bin of every audited instance, including expected and observed publication/output counts. The independent four-channel cases also check every published bin of each lane, for both the SIMD analyzer and four scalar native/first-party analyzers, using four distinct input fixtures. The reference uses direct DFT sums for $N \leq 256$ and an independently structured binary64 FFT for larger frames on the same rounded windowed input, followed by direct band sums and temporal averaging. Binary64 is not an arbitrary-precision oracle. For processed magnitude vectors a, r , both $\|a - r\|_2/\|r\|_2$ and $\|a - r\|_\infty/\|r\|_\infty$ must not exceed 3×10^{-4} for float

or 10^{-10} for double. Each denominator belongs to that frame and channel; a zero reference requires exactly zero output. Non-finite values fail. These are engineering budgets, not per-bin relative-error guarantees; weak-bin diagnostics remain available.

Periodic inverse and chain audits use the analytical inverse and direct time-domain references in Section 6, including checks of continuous chain delivery. Their pointwise budgets differ from the analysis norm budgets. Isolated transforms additionally check 17 spread-out direct-DFT outputs at the measured lengths and opposite-transform recovery; native adapters also compare all outputs against the first-party transform. These checks do not give every transform boundary the analysis audit’s all-bin independent-FFT reference. The campaign’s independent four-channel measurements have all-bin numerical audits; separate generic SIMD and module preflights are not substitutes for those measurements. Missing norms on inverse/chain or isolated-transform rows mean a different audit, not zero error. Transition replay verifies the request, applied state, first affected publication, and subsequent outputs under the recorded event schedule, rather than inserting them into the stationary rankings.

Every group passed source/dependency, numerical-coverage, and session checks. Portable bundles rederived all eight report tables in each group exactly, allowing only documented path relocation. Setup/storage probes remain outside timed execution. The compact committed tables and the publication generator support checking the manuscript without rerunning timings; regenerating the statistics from individual observations additionally requires the retained local raw bundles (Appendix K).

8 Comparative Results

The confirmation experiment supports a scheduling tradeoff, rather than a throughput advantage for the scalar resumable transform. Distributing the complete analysis reduces the concentration of work in the tested callbacks, but optimized native batch implementations generally have lower aggregate cost and publish the selected frame sooner. The inverse and complete-chain controls further delimit the result: distributing butterflies alone does not consistently beat an optimized batch transform’s callback duration. All results below come from the September 30 confirmation campaign. Historical measurements are not used to rank the current implementations.

8.1 Scalar analysis: cost, concentration, and age

Table 4 uses the middle of the three primary transform lengths, rather than selecting the length with the largest effect. The workload fixes the frame cadence, precision, required unsmoothed magnitudes, instance count, input bytes, and callback origin. At this configuration, the production core costs 34.98 ns per engine sample and has a callback p99 of 3.417 μ s. The native vDSP adapter costs 15.38 ns per sample with p99 9.500 μ s: the core has 2.27 times the aggregate cost while reducing this p99 statistic by a factor of 2.78. This is a practical implementation comparison, not the isolated cost of suspending a common FFT kernel.

The matched PFFFT scheduled-batch/hybrid pair gives a narrower attribution. Both retain the same transform arithmetic and surrounding scheduled tasks; the hybrid redistributes preparation and output work while leaving the native FFT indivisible. Its smaller p99 shows that distributing those surrounding stages already removes much of the callback concentration. The production core lowers p99 further at this configuration, but costs more than the hybrid. The legacy batch/incremental pair likewise shows reduced concentration, while its remaining unscheduled frame and output stages explain why it does not attain the production core’s p99. These controls separate the benefit

Implementation	Cost (ns/sample)	p99 (μ s)	Max. (μ s)	Delay
Production core	34.98 [34.75, 35.15]	3.42 [3.38, 3.46]	70.12	1023
Legacy batch	46.73 [46.18, 47.39]	42.92 [42.67, 42.96]	67.58	0
Legacy incremental	43.51 [43.24, 43.73]	8.46 [8.38, 8.54]	125.58	1023
PFFFT scheduled batch	36.63 [36.23, 37.22]	32.50 [32.42, 32.71]	75.42	0
PFFFT hybrid	31.42 [31.10, 31.68]	5.38 [5.38, 5.46]	41.79	1023
PFFFT native	15.67 [14.37, 18.27]	9.62 [9.54, 9.67]	108.38	0
FFTW native	16.32 [16.24, 16.46]	11.75 [11.54, 11.75]	40.58	0
vDSP native	15.38 [14.20, 16.24]	9.50 [9.42, 9.50]	33.29	0

Table 4: Scalar float analysis at the middle primary size $N = 4096$, $H = 1024$, $D = 64$, $f_s = 48$ kHz, one instance, no smoothing. Cost comes from the throughput pass; p99 and maxima come from the separate callback pass. Brackets are observed ranges of the three session means (cost) or session medians (p99); neither is a confidence interval. Maxima retain every observation from the nine callback processes. Delay is the frame-endpoint-to-publication delay in samples, before callback-end visibility.

of work placement from the much larger difference between scalar kernels and optimized native implementations.

Figure 1 retains all three primary lengths at a fixed hop and callback size. The core’s lower callback p99 coexists with higher aggregate cost than the native analysis adapters throughout these lengths. Fixed H makes longer transforms more work per publication; these curves are not an experiment with constant overlap ratio. The hybrid’s large throughput variation at the longest length is retained: its session means span 128.15–215.10 ns per sample. A single favorable run would misrepresent this observation.

Publication delay is the other material cost. At $H = 1024$ and 48 kHz, the production core and hybrid add 21.31 ms after the selected frame endpoint; immediate implementations add none. At $N = 4096$, the nominal spectrum-center age is therefore 63.97 ms for the core and 42.66 ms for immediate batch execution. Neither quantity includes callback-end visibility, repaint delay, or smoothing response. A visualizer that can accept the additional age and places high value on shorter callbacks has a reason to choose the core; an application primarily constrained by throughput or publication age has a reason to prefer native batch execution.

Observed maxima do not establish the same ranking as p99. For example, the core’s maximum at $N = 2048$ is 631.92 μ s despite a small session-median p99. The measurement includes interruptions on a general purpose operating system; these observations are retained, not attributed solely to the arithmetic scheduler. Conversely, the finite observations cannot certify that a more concentrated implementation meets every deadline. The logical credit bound and the measured duration distributions answer different questions.

8.2 Inverse jobs and complete frequency-domain chains

Table 5 keeps the periodic inverse job and both complete chains distinct. Relative to their matching scalar batch controls, the incremental controls substantially lower callback p99 while leaving aggregate cost of a similar order. Native implementations change that conclusion: for the periodic inverse workload at the representative size, PFFFT and vDSP have both lower aggregate cost and lower p99 than the incremental control. Thus the forward analyzer’s measured advantage in callback concentration does not generalize automatically to the inverse path.

The complete identity and FIR chains include retention, transforms, spectral multiplication,

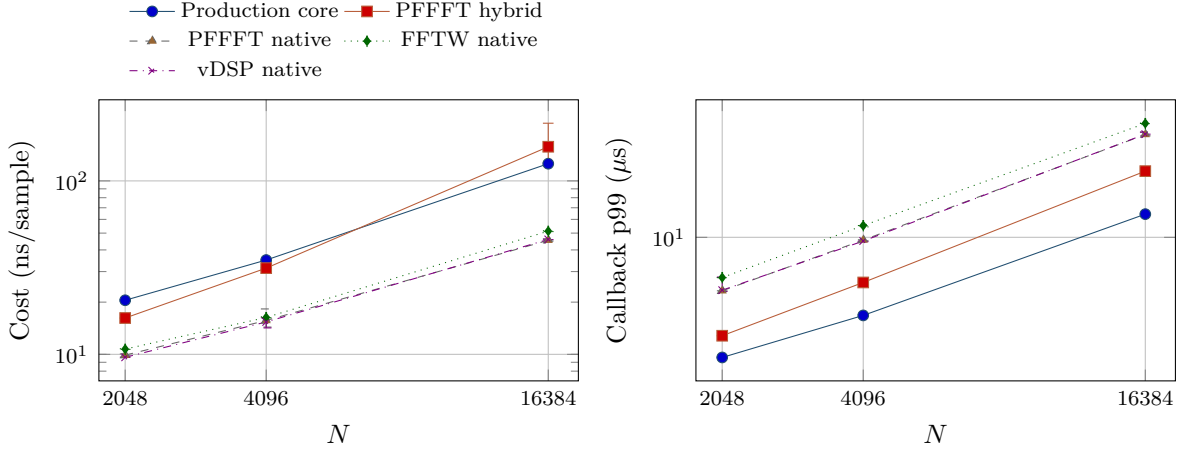


Figure 1: Primary scalar float length trends at $H = 1024$, $D = 64$, 48 kHz, without smoothing. Each point uses three sessions of three processes; bars show their full descriptive session range under the definitions of Table 4. The logarithmic axes distinguish aggregate work from callback concentration. No unfavorable process or session is removed.

normalization, overlap-save extraction, and output delivery. Their incremental variants have smaller p99 than the native chains at this configuration, at a large aggregate-work penalty and an additional hop-minus-one samples of delivery delay. These controls retain bulk input-buffering and frame-copy passes, including an $O(N)$ inverse-buffer task charged as one scheduling unit. They demonstrate a partially distributed frequency-domain operation, not the analyzer’s whole-pipeline work bound or a production audio effect.

Isolated execution reinforces the importance of kernel cost. At $N = 4096$, the normalized scalar IFFT costs $65.19 \mu\text{s}$ per transform, compared with $7.17 \mu\text{s}$ for vDSP under the isolated-transform contract. The isolated contract includes input buffering but excludes the periodic job’s per-sample scheduling and additional canonical-output copies. These values must not be interpreted as streaming cost per sample. The fixtures also do not justify choosing an FFT implementation for a three-tap filter, whose direct evaluation is a different, much smaller algorithmic workload.

8.3 Sensitivity, channels, and configuration changes

Table 6 tests the practical limits of the scalar analysis result. The p99 ordering depends on the hop, number and alignment of instances, and work performed around the analyzer. At the short $N = 128$, $H = 37$ setting, the native vDSP p99 is smaller than the core’s. Staggering many native analyses across callbacks can also reverse an advantage seen with synchronized publications. This is an alternative means of reducing aggregate callback concentration when the application controls frame phases. The independent small-hop campaign has a different observation window and is not pooled with the primary campaign.

Live window and smoothing updates give another limit: the hybrid’s p99 can be lower than the production core’s, even though the core is lower in the clean-cache reference case. The configured background DSP brings the total callback statistics closer because that shared work occupies more of the interval. Higher sample rate changes the callback budget and sample-index age without inherently reducing the work in a fixed-length transform. None of these extensions measures a complete Rack patch or an audio device.

Single-sample callbacks illustrate a percentile failure rather than an especially favorable scheduler.

Workload	Implementation	Cost (ns/sample)	p99 (μ s)	Delay
Inverse job	Batch control	79.15 [77.80, 80.09]	78.25 [76.83, 78.83]	0
	Incremental control	72.55 [72.00, 73.14]	10.17 [10.12, 10.21]	1023
	PFFFT	12.94 [12.89, 13.00]	8.25 [8.21, 8.29]	0
	FFTW	15.05 [15.01, 15.07]	10.42 [10.42, 10.42]	0
	vDSP	12.85 [12.81, 12.92]	7.79 [7.79, 7.79]	0
Identity chain	Batch control	150.25 [149.91, 150.44]	150.12 [149.58, 151.25]	1023
	Incremental control	152.12 [150.42, 154.34]	17.00 [16.71, 17.58]	2046
	PFFFT	25.79 [25.28, 26.67]	20.79 [20.71, 20.79]	1023
	FFTW	29.50 [29.27, 29.64]	25.04 [25.00, 25.08]	1023
	vDSP	25.42 [24.95, 26.05]	20.17 [20.08, 20.46]	1023
FIR chain	Batch control	152.89 [150.58, 156.47]	150.17 [149.62, 150.62]	1023
	Incremental control	153.83 [151.41, 156.45]	17.21 [17.00, 17.58]	2046
	PFFFT	25.33 [25.07, 25.50]	20.92 [20.79, 21.00]	1023
	FFTW	29.55 [29.32, 29.74]	25.00 [24.92, 25.50]	1023
	vDSP	25.04 [24.83, 25.21]	20.21 [20.21, 20.29]	1023

Table 5: Periodic inverse jobs and complete overlap-save chains, with the same N, H, D, f_s , precision and instance count as Table 4. Session summaries use the same definitions. Delay is release-to-publication for inverse jobs and input-to-delivery for chains (samples). The incremental controls distribute butterflies but retain bulk buffering and copy stages; they do not inherit the production analyzer’s whole-pipeline quota bound. The identity and three-tap FIR are functional fixtures, not efficient application choices for FFT convolution.

For $N = 2048$, $H = 1024$, the core and native vDSP both have p99 0.042μ s, equal to the median empty-timer p99 of 0.042μ s, while their retained maxima are 8.75μ s and 9.12μ s. One transform release per 1024 samples can fall outside p99. These short, unpaced experiments retain only 64 hops per process; they cannot identify a precise rare-event tail from the apparent equality of p99.

Table 7 prevents the scalar result from standing in for the independent-channel implementation. The four-lane core substantially reduces aggregate work relative to four scalar cores. At the selected size, its aggregate cost is also below the measured banks of native float analyses, while retaining a smaller p99. This is evidence for this channel-parallel implementation and input layout on the measured host, not a general SIMD advantage over those libraries. The double-precision rows form their own comparison and retain the same throughput/concentration tradeoff.

The transition suite checks requests, application boundaries, publication responses, and every affected output together (Table 8). Across both transition horizons, 11889 published spectra were audited. Requests can replace earlier requests, and a final request can remain pending at the finite horizon; neither outcome is silently converted into a successful response. The displayed response delay can exceed a stationary hop because it begins at the request and includes latching and changed-hop scheduling. Event-window maxima include overlap between adjacent request windows and ordinary system interference, so subtracting the no-change maximum would not isolate the cost of a settings change.

8.4 Numerical and resource evidence

Every analysis coverage record has its expected publication and bin counts. Table 9 reports the largest observed vectorwise errors separately by precision, including the independent channels. The recorded float and double maxima satisfy their frozen normwise acceptance budgets established before confirmation. The nonzero float pointwise diagnostics remain visible: passing a spectrum

Configuration	Production core	PFFFT hybrid	vDSP native
Reference	1.92 [1.92, 1.92]	2.58 [2.58, 2.58]	4.83 [4.75, 5.79]
Smoothing	2.17 [2.17, 2.21]	2.62 [2.62, 2.67]	7.17 [5.83, 7.17]
Shorter hop	4.88 [4.75, 5.08]	5.29 [5.25, 5.33]	5.04 [4.83, 6.00]
Live cache/settings	17.12 [16.75, 17.83]	13.92 [13.88, 13.92]	26.42 [24.58, 28.33]
Startup	1.92 [1.88, 1.96]	2.62 [2.58, 2.62]	4.83 [4.75, 4.83]
96 kHz	1.92 [1.88, 1.96]	2.62 [2.58, 2.67]	5.04 [4.75, 5.08]
Background load 64	10.21 [9.96, 10.33]	8.29 [8.17, 8.50]	10.33 [10.33, 11.62]
32 MiB cache touch	4.33 [4.12, 4.79]	5.83 [5.42, 6.25]	7.79 [7.29, 9.33]
Callback offset 1	1.92 [1.88, 1.96]	2.62 [2.58, 2.71]	4.88 [4.79, 5.54]
Callback offset 15	1.92 [1.88, 1.96]	2.62 [2.62, 2.67]	4.88 [4.75, 4.88]
Callback offset 63	1.92 [1.92, 2.00]	2.67 [2.62, 2.67]	5.04 [4.79, 5.79]
4 aligned instances	8.38 [7.96, 8.58]	11.04 [11.00, 11.04]	21.67 [21.21, 21.88]
4 staggered instances	6.33 [6.08, 6.33]	5.62 [5.58, 5.71]	6.75 [6.71, 7.04]
16 aligned instances	38.33 [33.25, 41.00]	45.12 [45.12, 45.29]	86.38 [85.96, 89.25]
16 staggered instances	28.71 [25.08, 34.71]	19.58 [18.29, 21.08]	10.33 [10.25, 10.33]
$N = 128, H = 37$	1.79 [1.79, 1.92]	1.79 [1.79, 1.88]	0.92 [0.92, 0.92]

Table 6: Sensitivity of callback p99 in microseconds (session range in brackets). Defaults are scalar float, $N = 2048$, $H = 1024$, $D = 64$, 48 kHz, one aligned instance, no smoothing/load/cache touch, zero callback offset. Only the named factors change. Shorter-hop and live-cache rows use $H = 257$; the latter also enables smoothing. Rows are separate matched comparisons; their different durations and observation counts are not pooled. Background load 64 is a cascade of 64 binary64 DC blockers, included in timed work; cache touching occurs outside timing. The three displayed implementations span fully distributed, hybrid and native execution; the complete evidence retains the remaining backends.

norm does not bound the relative error of every weak bin. The transition audits also pass; their largest relative ℓ_∞ errors are 8.16×10^{-7} and 6.01×10^{-15} for float and double respectively.

Inverse and chain checks use their own analytical/time-domain references and output-amplitude units. Their largest recorded absolute errors across all stationary configurations are 5.50×10^{-7} and 6.01×10^{-7} for float, and 3.66×10^{-15} and 6.00×10^{-16} for double, respectively. These maxima do not replace the reference-specific acceptance tests or extend a partial isolated transform check to every possible spectrum.

Storage probes require equally careful scope. At the representative scalar analysis configuration, the recorded C++ live-heap counters are 556.5 KiB for the core and 136.6 KiB for the native vDSP adapter. These cover intercepted C++ heap allocations but exclude unknown native-library storage, stack workspace and allocator overheads; they cannot establish a total-memory ranking. In particular, observing no intercepted C++ allocation during execution does not exclude allocation inside an opaque library. The archived provider plans and allocation-scope records retain these distinctions.

Configuration	Cost (ns/engine sample)	p99 (μ s)
Four scalar cores	147.06 [141.31, 156.20]	14.08 [13.58, 14.12]
Four-lane SIMD core	47.85 [47.47, 48.05]	4.58 [4.50, 4.58]
Four PFFFT analyses	52.94 [51.90, 53.85]	39.08 [38.17, 40.00]
Four FFTW analyses	62.02 [60.42, 63.19]	47.42 [46.67, 47.92]
Four vDSP analyses	57.27 [56.09, 59.35]	43.92 [42.29, 44.04]
Scalar double core	36.23 [35.85, 36.63]	4.46 [4.38, 4.50]
Scalar double FFTW	19.90 [19.70, 20.05]	15.21 [14.92, 15.83]
Scalar double vDSP	18.80 [18.53, 18.99]	14.21 [13.96, 14.54]

Table 7: Independent-channel and precision comparisons at $N = 4096$, $H = 1024$, $D = 64$, 48 kHz without smoothing. The first five rows include four independent float channels in every engine sample; the last three contain one double channel. Each block is comparable internally, not across channel counts. Double adapters receive the common rounded float input samples and window coefficients promoted separately, with their product evaluated in double.

Implementation	Response (ms)	Control max. (μ s)	Change max. (μ s)
Production float	85.27	22.38	33.21
Production double	85.27	31.17	39.50
PFFFT hybrid	85.27	10.88	68.75
PFFFT scheduled batch	42.62	22.29	226.04

Table 8: Interactive transition suite with initial $N = 2048$, $H = 1024$, $D = 64$, $f_s = 48$ kHz and 65,536-sample horizon. Response is the largest request-to-first-publication delay among responding requests, not the stationary frame age. Control/change columns retain the largest callback in any event window and any of nine processes, without pooling or subtraction. Every process has nine published requests, one replaced request and one pending request at the finite horizon. The separate short-hop transition suite is retained in the evidence; window overlaps and small counts preclude a rare-event tail estimate.

Precision	Audited spectra	Maximum relative ℓ_2	Maximum relative ℓ_∞	Diagnostics
float	11,658,240	4.77×10^{-7}	6.47×10^{-7}	370,800
double	1,013,760	5.96×10^{-15}	6.02×10^{-15}	0

Table 9: Stationary analysis audit totals and largest observed per-vector normwise errors over all four stationary groups, passes, sessions, processes, instances and channels. These repeated vector checks are coverage counts, not independent statistical replicates. Diagnostics count violations of the retained legacy pointwise rule; that rule is not the frozen spectrum-norm acceptance test. Zero-reference vectors are checked separately. Inverse and chain absolute-error checks have different output units and are not mixed into these norms.

9 Historical Evidence and Scope

Two archived campaigns explain the design’s evolution. The original scalar FFT study compares immediate and incremental execution at fixed requested cadence. It locates residual preparation/reconstruction bursts and shows why p99 can miss work occurring less than once per hundred sample calls. Appendices F and G preserve its methods, within-session resampling, and complete numerical and timing results.

The complete-pipeline prototype compares a serial one-hop schedule with boundary bursts and four-hop overlap (Appendix H). It illustrates lower observed block maxima alongside higher aggregate cost; the one-hop design also avoids the overlapped prototype’s additional $3H$ samples of age. Some controls change storage and arithmetic as well as placement. These historical results do not rank the current weighted analyzer or optimized libraries. The confirmation study supplies that separate evidence; neither historical nor current campaigns measure a running audio device or concurrent display consumption.

10 Discussion and Limits

10.1 When the tradeoff is useful

The strongest case for cooperative analysis is a caller that values reducing concentrated engine work and can tolerate an additional hop of spectrum age. The representative comparison reduces callback p99 while increasing aggregate cost. That is useful evidence for an engineering choice, not a new transform complexity result. For a low-cost visualizer, headroom in the callback budget may matter more than minimum arithmetic throughput; an application needing fresh output or many concurrent analyzers may make the opposite choice. The observed timing distributions cannot decide those application priorities. For the representative $D = 64$, 48 kHz case, the nominal callback interval is $1333.33 \mu\text{s}$: the production and vDSP p99 values occupy only 0.26% and 0.71% of that interval, respectively. Thus the measured reduction does not show that the native baseline was near its deadline, or that redistribution prevented audible failures. Such a claim would require a contended host and device measurements.

The PFFFT hybrid is a useful intermediate design: it distributes surrounding work while keeping a native FFT indivisible. It shows that scheduling the pipeline around a batch transform can capture some benefit without suspending individual butterflies. Its comparison with the matched scheduled-batch control isolates work placement more closely than a comparison with the production analyzer, which also changes arithmetic, layout, storage, and caching. The extra state in a resumable transform is justified only when its measured behavior meets a requirement that the simpler alternative does not.

The inverse and chain results sharpen that qualification. Deferring butterfly execution does not eliminate the bulk buffering and conjugation at job or stage boundaries. The representative native PFFFT and vDSP inverse jobs provide both lower aggregate cost and lower p99 than the incremental control. The complete-chain control reduces p99 more modestly than its large aggregate-cost penalty. Thus the favorable analysis example cannot be generalized to all frequency-domain processing. The formulas in Section 6 identify what a future complete inverse scheduler would need to change. The current measurements are already useful as counterexamples to a blanket claim that incremental execution reduces callbacks more effectively than optimized batch execution.

10.2 Interpretation and external validity

The primary tables compare declared input/output contracts. Smoothing interval arithmetic and transform layouts differ across implementation families; numerical agreement is within the stated contracts and budgets, not bitwise identity. Precision, channel count, state, callback phase, and additional DSP change the task. Extension results should be read within each matched group rather than combined into a single provider score.

A high percentile, a maximum, and aggregate throughput answer different questions. Single-sample p99 can omit frame work occurring less than once per hundred calls. Maxima retain those bursts and rare preemptions but depend on observation duration; the separate group durations preclude indiscriminate maxima comparisons. No samples are trimmed and no empty-timer cost is subtracted. Small absolute differences and sub-tick observations should not be interpreted as finely resolved instruction costs. Storage probes measure only their declared allocation scope; unknown provider-native storage prevents a complete memory ranking.

All confirmation observations come from one compiler and one macOS ARM64 host on one night. The prepared sessions expose temporal variation, including unfavorable session excursions, but do not establish independent-host replication or a stable rare-event distribution. Dynamic FFTW planning is part of the recorded configuration. Other architectures, compilers, and plans may change the relative costs. Binary64 references, deterministic fixtures, and all-publication audits support the stated numerical workloads, not every possible input or application-specific tolerance.

10.3 Work bounds and real-time behavior

The proof bounds credits in the complete analysis sequence. Unequal unit costs, memory hierarchy, preemption, host lifecycle work, and unrelated DSP still determine elapsed time. A late call executes its logical quota; the scheduler does not recover a missed deadline. Producer publication also does not guarantee display consumption or repaint cadence: a slow display may skip complete snapshots by design.

The benchmark runs synchronously without an audio driver or concurrent UI. It therefore provides no measured device underrun rate or proof of hard real-time safety. Worker-thread analyzers, overlap-reusing sliding/hopping methods, non-power-of-two transforms, and long-filter applications are not ranked. Appendix J identifies the next experiments. The complete paper can support an implementation decision under its recorded conditions; broader recommendations require those additional measurements.

11 Conclusion

A complete windowed analyzer can be expressed as bounded, dependency-ordered operations and advanced over an exact publication hop. The resulting contract accounts for work credits, retained input, cancellation, and ownership of complete spectra. It addresses the surrounding preparation and output stages that remain concentrated when only FFT butterflies are resumable.

The confirmation study establishes a practical tradeoff on the measured M1 Pro host: cooperative analysis can reduce callback p99 at the expense of total cost and spectrum age. Matched controls distinguish that placement effect from the benefits of optimized kernels, and the native PFFFT hybrid provides an intermediate option. Periodic inverse jobs and overlap-save chains retain bulk stages and include cases favoring native batch processing; their results do not inherit the complete analyzer’s bound. The value of suspension is therefore conditional on the full processing path and the caller’s priorities.

References

- [1] J. W. Cooley and J. W. Tukey, “An algorithm for the machine calculation of complex Fourier series,” *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965. doi:10.1090/S0025-5718-1965-0178586-1.
- [2] H. V. Sorensen, D. L. Jones, M. T. Heideman, and C. S. Burrus, “Real-valued fast Fourier transform algorithms,” *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 35, no. 6, pp. 849–863, 1987. doi:10.1109/TASSP.1987.1165220.
- [3] J. B. Allen and L. R. Rabiner, “A unified approach to short-time Fourier analysis and synthesis,” *Proceedings of the IEEE*, vol. 65, no. 11, pp. 1558–1564, 1977. doi:10.1109/PROC.1977.10770.
- [4] F. J. Harris, “On the use of windows for harmonic analysis with the discrete Fourier transform,” *Proceedings of the IEEE*, vol. 66, no. 1, pp. 51–83, 1978. doi:10.1109/PROC.1978.10837.
- [5] P.-C. Lo and Y.-Y. Lee, “Real-time implementation of the split-radix FFT—an algorithm to efficiently construct local butterfly modules,” *Signal Processing*, vol. 71, no. 3, pp. 291–299, 1998. doi:10.1016/S0165-1684(98)00152-2.
- [6] P.-C. Lo and Y.-Y. Lee, “Real-time FFT algorithm applied to on-line spectral analysis,” *Circuits, Systems, and Signal Processing*, vol. 18, no. 4, pp. 377–393, 1999. doi:10.1007/BF01200789.
- [7] P.-C. Lo and Y.-Y. Lee, “Real-time implementation of the moving FFT algorithm,” *Signal Processing*, vol. 79, no. 3, pp. 251–259, 1999. doi:10.1016/S0165-1684(99)00098-5.
- [8] C. Borß, “A VST reverberation effect plugin based on synthetic room impulse responses,” in *Proceedings of the 12th International Conference on Digital Audio Effects (DAFx-09)*, Como, Italy, September 2009. https://www.dafx.de/paper-archive/2009/papers/paper_48.pdf.
- [9] J. Hurchalla, “A time distributed FFT for efficient low latency convolution,” in *129th Audio Engineering Society Convention*, paper 8257, 2010. <https://secure.aes.org/forum/pubs/conventions/?elib=15679>.
- [10] E. Battenberg and R. Avizienis, “Implementing real-time partitioned convolution algorithms on conventional operating systems,” in *Proceedings of the 14th International Conference on Digital Audio Effects (DAFx-11)*, Paris, France, September 2011. <https://ericbattenberg.com/pdf/partconvDAFx2011.pdf>.
- [11] F. Wefers, *Partitioned Convolution Algorithms for Real-Time Auralization*, Aachener Beiträge zur Technischen Akustik, vol. 20. Berlin: Logos Verlag, 2015. Ph.D. dissertation, RWTH Aachen University, defended 2014. doi:10.18154/RWTH-2015-02185.
- [12] J. Liu, B. Yan, and Q. Zou, “Optimal time-distributed fast Fourier transform: Application to online iterative learning control—experimental high-speed nanopositioning example,” *Mechatronics*, vol. 41, pp. 114–124, 2017. doi:10.1016/j.mechatronics.2016.11.007.
- [13] Z. Průša and N. Holighaus, “Real-time audio visualization with reassigned non-uniform filter banks,” in *Proceedings of the 19th International Conference on Digital Audio Effects (DAFx-16)*, Brno, Czech Republic, pp. 3–8, September 2016. https://www.dafx.de/paper-archive/2016/dafxpapers/01-DAFx-16_paper_20-PN.pdf.
- [14] E. Jacobsen and R. Lyons, “The sliding DFT,” *IEEE Signal Processing Magazine*, vol. 20, no. 2, pp. 74–80, 2003. doi:10.1109/MSP.2003.1184347.
- [15] R. Lyons and C. Howard, “Improvements to the sliding discrete Fourier transform algorithm,” *IEEE Signal Processing Magazine*, vol. 38, no. 4, pp. 119–127, 2021. doi:10.1109/MSP.2021.3075416.

- [16] L. F. Richardson and W. F. Eddy, “Algorithm 991: The 2D tree sliding window discrete Fourier transform,” *ACM Transactions on Mathematical Software*, vol. 45, no. 1, article 12, 2019. doi:10.1145/3264426. Preprint: <https://arxiv.org/abs/1707.08213>.
- [17] M. Garrido, “The feedforward short-time Fourier transform,” *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 63, no. 9, pp. 868–872, 2016. doi:10.1109/TCSII.2016.2534838.
- [18] C.-S. Park and S.-J. Ko, “The hopping discrete Fourier transform,” *IEEE Signal Processing Magazine*, vol. 31, no. 2, pp. 135–139, 2014. doi:10.1109/MSP.2013.2292891.
- [19] Z. Rafii, “Sliding discrete Fourier transform with kernel windowing,” *IEEE Signal Processing Magazine*, vol. 35, no. 6, pp. 88–92, 2018. doi:10.1109/MSP.2018.2855727.
- [20] C. Eleftheriadis, M. Garrido, and G. Karakostas, “Energy-efficient short-time Fourier transform for partial window overlapping,” in *Proceedings of the IEEE International Symposium on Circuits and Systems (ISCAS)*, 2023. doi:10.1109/ISCAS46773.2023.10181940.
- [21] A. Bécoulet and A. Verguet, “A depth-first iterative algorithm for the conjugate pair fast Fourier transform,” *IEEE Transactions on Signal Processing*, vol. 69, pp. 1537–1547, 2021. doi:10.1109/TSP.2021.3060279.
- [22] M. Frigo and S. G. Johnson, “The design and implementation of FFTW3,” *Proceedings of the IEEE*, vol. 93, no. 2, pp. 216–231, 2005. doi:10.1109/JPROC.2004.840301. Planner API: https://www.fftw.org/fftw3_doc/Planner-Flags.html (accessed September 30, 2026).
- [23] J. Pommier and contributors, “PFFFT: A pretty fast FFT,” software; VCV Rack integration, `dsp::RealFFT`. Accessed September 30, 2026. <https://github.com/VCVRack/pffft>. Rack FFT wrapper source.
- [24] Apple, “Using Fourier Transforms,” *vDSP Programming Guide*, Accelerate framework documentation. Accessed September 30, 2026.
- [25] A. van der Byl and M. R. Inggs, “Constraining error—a sliding discrete Fourier transform investigation,” *Digital Signal Processing*, vol. 51, pp. 54–61, 2016. doi:10.1016/j.dsp.2016.01.008.
- [26] T. Skare, “General-purpose GPU audio benchmark framework,” in *Proceedings of the 27th International Conference on Digital Audio Effects (DAFx24)*, Guildford, United Kingdom, pp. 452–458, September 2024. https://www.dafx.de/paper-archive/2024/papers/DAFx24_paper_56.pdf.
- [27] D. R. K. Balasubramaniam, S. Ramachandran, and J. Timoney, “Real-time neural audio on Apple Silicon: Benchmarking inference frameworks under realistic DAW contention,” in *Proceedings of the 29th International Conference on Digital Audio Effects (DAFx26)*, Cambridge, MA, USA, pp. 128–135, September 2026. https://www.dafx.de/paper-archive/2026/papers/DAFx26_paper_16.pdf.
- [28] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, “Producing wrong data without doing anything obviously wrong!,” in *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS ’09)*, pp. 265–276, 2009. doi:10.1145/1508244.1508275.
- [29] T. Kalibera and R. Jones, “Rigorous benchmarking in reasonable time,” in *Proceedings of the 2013 International Symposium on Memory Management (ISMM ’13)*, pp. 63–74, 2013. doi:10.1145/2464157.2464160. Corrected author manuscript: <https://kar.kent.ac.uk/33611/>.
- [30] R. Wilhelm et al., “The worst-case execution-time problem—overview of methods and survey of tools,” *ACM Transactions on Embedded Computing Systems*, vol. 7, no. 3, article 36, 53 pages, 2008. doi:10.1145/1347375.1347389.
- [31] VCV, “Plugin API guide,” *VCV Rack Manual*. Accessed September 28, 2026. <https://vcvrack.com/manual/PluginGuide>.

A Transform Derivations and Resumable Traversal

The frame and normalization convention is defined in Section 3. This appendix records the conventional transform mathematics and the original state transition used by the reusable FFT classes; the complete analysis schedule is in Section 5.

A.1 Resumable complex transform

For $N = 2^p$, bit reversal places the input in the order required by an iterative radix-2 decimation-in-time transform. A stage of span $s = 2, 4, \dots, N$ visits groups $g = 0, s, 2s, \dots, N - s$ and pairs $a = 0, \dots, s/2 - 1$. One step computes

$$e = c[g + a], \quad o = W_N^{aN/s} c[g + a + s/2], \quad (11)$$

$$c[g + a] \leftarrow e + o, \quad c[g + a + s/2] \leftarrow e - o, \quad (12)$$

where $W_N = \exp(-j2\pi/N)$. Advancing a , then g , then s defines a serial traversal of the conventional FFT graph. Retaining these three indices is sufficient to suspend and resume that traversal. Each stage has $N/2$ butterflies, so the total count is

$$B_c(N) = \frac{N}{2} \log_2 N. \quad (13)$$

The coefficient array, twiddle table, and bit-reversal table each occupy $O(N)$ storage; the traversal state occupies $O(1)$ storage. Buffer preparation copies and permutes the entire frame. Resumability applies to the subsequent butterfly traversal, not automatically to that preparation.

Proposition 1 (Arithmetic-order preservation). *For a fixed buffered frame and transform configuration, any partition of the serial butterfly traversal into finite batches yields the same completed coefficient array as uninterrupted traversal, provided every butterfly executes exactly once in the original order and no external code mutates the state.*

Proof. After buffering, the two traversals have identical coefficients and indices. If they agree before a butterfly, Equation (12) reads the same operands and performs the same updates; the index transition also agrees. Induction over B_c establishes equality. A pause changes no state. In floating point, bitwise equality additionally assumes the same arithmetic instructions, rounding environment, and compiler transformations. The experiments test bitwise equality within a single executable rather than asserting it across different compilers or FFT libraries. $\square$

A.2 Real-input packing and reconstruction

For even N , define $M = N/2$ and pack $z[m] = u[2m] + ju[2m + 1]$. Let Z be its M -point complex transform. For $1 \leq k < M$, with $A = Z[k]$ and $D = \overline{Z[M - k]}$, reconstruction is

$$X[k] = \frac{1}{2}(A + D - jW_N^k(A - D)), \quad X[N - k] = \overline{X[k]}. \quad (14)$$

The endpoints are $X[0] = \Re Z[0] + \Im Z[0]$ and $X[M] = \Re Z[0] - \Im Z[0]$. This follows by separating the transforms of the even and odd real samples; it is a standard consequence of conjugate symmetry, not a new real FFT.

```

void step() {
    if (span > N) return;
    const size_t half = span / 2;
    const auto even = c[group + pair];
    const auto odd = multiply(c[group + pair + half],
                             twiddle[pair * (N / span)]);

    c[group + pair] = even + odd;
    c[group + pair + half] = even - odd;
    if (++pair == half) {
        pair = 0;
        group += span;
        if (group == N) { group = 0; span *= 2; }
    }
}

```

Listing 1: Simplified state transition corresponding to the original complex FFT. Tables and storage are prepared separately.

The resumable inner transform performs

$$B_r(N) = B_c(N/2) = \frac{N}{4} \log_2(N/2) \quad (15)$$

butterflies. The original real-FFT class materializes all N output bins, including conjugate copies, and reconstructs them in one $O(N)$ loop when the last inner butterfly finishes. Thus a real-FFT `step()` is not uniformly a single butterfly: the completing step also performs reconstruction. The operational distinction is essential to the timing model in Appendix C.

The inverse class uses $\text{IFFT}(X) = \overline{\text{FFT}(\overline{X})}/N$. It treats each output normalization as an additional resumable step, giving $B_c(N) + N$ steps after buffering. This differs from the real forward transform’s bulk reconstruction. Input copying, permutation, and conjugation remain $O(N)$ buffering work. Neither production visualization module uses the inverse transform. The original historical campaign concerns the real forward path; the replacement campaign also measures isolated inverse transforms, periodic inverse jobs, and complete filtering chains. Their normalization, buffering, and delivery contracts are stated in Section 6.

B Original Scheduling, Cadence, and Latency

Let $H \geq 1$ be the requested computation horizon in sample calls and let $B > 0$ be the number of inner butterflies. The implementation chooses

$$q = \lceil B/H \rceil \quad (16)$$

and invokes the single-step method q times per call. Once complete, further step invocations are no-ops. The following statements concern this integer model. These reusable forward classes compute the ceiling through single-precision floating-point division; the reported experiments verify the model over the tested range. Extremely large sizes or horizons require a separate analysis of rounding and integer overflow. Zero horizons and unsupported transform lengths are outside the forward-path contract examined here. We use powers of two $N \geq 4$ for the real transform.

Proposition 2 (Completion bound). *The transform completes after exactly $L = \lceil B/q \rceil$ calls, with $L \leq H$.*

Proof. After ℓ calls the number of completed butterflies is $\min(B, \ell q)$. The least ℓ reaching B is $\lceil B/q \rceil$. Since $qH \geq B$, this value cannot exceed H . $\square$

This is a bound on operation counts, not execution time. In particular, completion *within* H calls does not imply completion *at* call H . In the original snapshot, both visualization modules test completion at the start of their processing routine, publish the previous result, buffer a new frame, and then advance the new transform. In steady state, with constant parameters and active processing, frame starts are therefore separated by L samples rather than necessarily by H samples. Initialization, freeze, reset, and reconfiguration are separate state transitions and are not covered by this steady-state statement.

For $N = 4096$ and $H = 2048$, $q = 6$ and $L = 1878$. At 48 kHz the nominal interval is 42.67 ms, whereas the original steady-state interval is 39.125 ms. The rate difference is about 9.1%. This is not a numerical FFT error; it is a difference between the parameter’s nominal interpretation and the scheduler’s completion-driven frame clock.

B.1 Timestamp semantics

Suppose a complete input window is captured on call t_r and its first batch runs in that same call. Its final batch runs at $t_r + L - 1$, and the module publishes the result at the start of call $t_r + L$. Measured from the newest sample, publication delay is consequently L/f_s . Relative to the midpoint of the captured sample support, its age is

$$A_{\text{mid}} = \frac{(N - 1)/2 + L}{f_s}. \quad (17)$$

The midpoint is a timestamp convention, not a universal group-delay claim for arbitrary windows. The display thread may add further delay. Figure 2 separates the sample support from the deferred calculation. Streaming input continues during calculation, but those newer samples belong to a later frame.

B.2 Consequences for temporal smoothing

The original modules use an exponential magnitude smoother. Writing its intended time constant as τ and assuming a nominal interval H/f_s gives

$$y_r = \alpha y_{r-1} + (1 - \alpha)|X_r|, \quad \alpha = \exp[-H/(f_s \tau)]. \quad (18)$$

The implementation’s user-facing smoothing-time parameter corresponds to 10τ . If actual updates arrive every L/f_s , then after elapsed time t the old state is attenuated approximately by

N	$H = N/2$	B_r	q	L	H/L
1,024	512	2,304	5	461	1.111
2,048	1,024	5,120	5	1,024	1.000
4,096	2,048	11,264	6	1,878	1.091
16,384	8,192	53,248	7	7,607	1.077

Table 10: Derived real-FFT schedules. H/L is the actual frame rate divided by the nominal rate when a new frame starts immediately after completion. These are operation counts, not timing measurements.

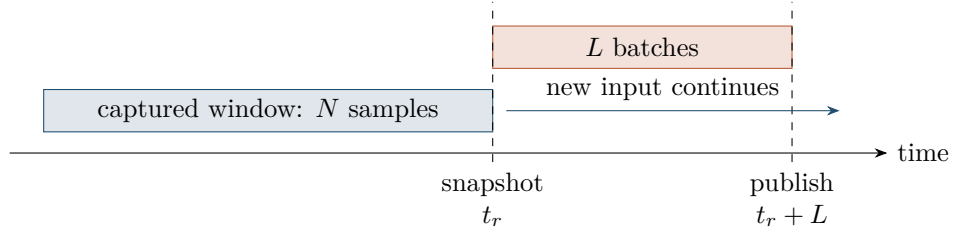


Figure 2: Logical timeline, not to scale. The transform operates on an owned copy of the captured frame while the input ring continues to advance. A smaller batch quota delays publication without changing that frame’s spectral contents.

$\exp[-tH/(L\tau)]$. The effective time constant is thus

$$\tau_{\text{eff}} = \tau L/H. \quad (19)$$

For the 4096-point example, the decay time is approximately 8.3% shorter than its nominal interpretation. The one-hop successor removes this completion-driven discrepancy; the Fourier module still truncates its hop control to integer samples (Section 5). The analysis also motivates timestamping spectrogram columns by captured frames rather than assuming the configured horizon is their spacing.

B.3 Exact-horizon alternative

If an application requires publication exactly every H calls, it can separate the frame clock from completion and retain a finished result until its scheduled publication. Alternatively, apply Equation (4) to the B ordered butterfly units in place of the complete pipeline’s W units. The quotas sum to B by telescoping; each is either $\lfloor B/H \rfloor$ or $\lceil B/H \rceil$. For $B > 0$, the last quota is positive, so the ordered work ends on call H . Among schedules for B unit-cost operations in H calls, no schedule can have a maximum quota smaller than $\lceil B/H \rceil$, by the pigeonhole principle. Equation (4) attains that lower bound. This elementary balanced schedule is not a claimed new scheduling result. The successor applies it to a larger work sequence (Section 5). Its unit-cost premise does not make unequal operations interchangeable in execution time.

C Original Whole-Pipeline Cost and Ownership

The real FFT’s public interface exposes a useful but partial work budget. A schematic call-cost model is

$$C_s = C_{\text{input},s} + C_{\text{dispatch},s} + b_s C_{\text{butterfly},s} + \mathbf{1}_{s=0} C_{\text{prepare}} + \mathbf{1}_{s=L-1} C_{\text{reconstruct}} + C_{\text{publish},s}. \quad (20)$$

Here $b_s \leq q$ is the number of useful butterflies, and costs need not be constant across stages or calls. Publication of a previous result may coincide with preparation of the next. The expression describes locations of work, not a hardware-independent cycle prediction.

In the source snapshot, real-input packing creates a temporary vector; copying and bit reversal are bulk operations; reconstruction is bulk; optional fractional-octave smoothing allocates a prefix-sum array; temporal smoothing and display-coordinate conversion visit complete arrays. Window-length reconfiguration may resize storage on the engine path. The existing implementation therefore

does not satisfy an allocation-free, uniformly small-call contract merely because its inner FFT is resumable.

If $H = \rho N$ for fixed $\rho > 0$, then $q = \Theta(\log N)$ for the real FFT. The butterfly work assigned to ordinary calls grows logarithmically, while preparation and reconstruction still contribute $\Theta(N)$ operations to boundary calls. Consequently, the full real-FFT path retains a linear peak operation count under this scaling. The change from a full $\Theta(N \log N)$ burst to smaller inner batches can still be useful, but the asymptotic distinction must be stated at the correct boundary.

C.1 Magnitude processing and SIMD

Fractional-octave smoothing uses a prefix sum of magnitudes, allowing a range mean to be evaluated with two prefix accesses after a linear setup pass. For fraction β of an octave, the nominal band around f_k is $[f_k 2^{-\beta/2}, f_k 2^{\beta/2}]$, discretized to FFT-bin indices and adjusted near Nyquist. The result is a visualization-oriented magnitude average, not a constant-Q transform or a power-conserving spectral estimate. Phase is discarded when smoothed magnitudes replace complex coefficients.

The Fourier module instantiates arithmetic on `simd::float_4`. Each lane carries an independent input channel through the same traversal; the SIMD lanes are not four consecutive bins of one transform. This arrangement shares index/control work across channels. It does not establish a measured fourfold speedup, and the scalar experiments in Appendix F do not validate Rack SIMD instantiations. Rack’s plugin API supplies the host processing and SIMD context [31].

C.2 Snapshots and publication

The input ring can continue advancing because `buffer()` copies the selected frame into transform-owned storage. Resuming directly from a mutable ring without preserving the selected samples would violate Equation (2). Any attempt to distribute packing must therefore solve input lifetime as well as arithmetic scheduling.

Similarly, a display-readiness flag is not by itself an engine-to-display synchronization protocol. Concurrent reads of arrays being modified require an ownership and publication design consistent with the C++ memory model. The original source’s shared display buffers are not certified by its scalar tests. A bounded exchange of preallocated buffers needs an explicit slow-consumer policy and safe reuse; an atomic pointer alone is insufficient. The successor’s ownership exchange is described in Section 5.

D One-Hop Scheduling and Implementation Details

Section 5 states the schedule and its observable contract. This appendix preserves the dispatch algorithm, proof, worked quotas, input lifetime argument, and module-specific processing details.

D.1 Balanced dispatch and worked quotas

Implement Equation (4) with a quotient/remainder accumulator. Quotient and remainder $a = \lfloor W/H \rfloor$ and $r = W \bmod H$ are refreshed at configuration, reset, and the transition from a rebuilt window to a clean cache. Starting the accumulator e at zero, each sample consumes a credits plus one when $e + r \geq H$, then retains $(e + r) \bmod H$:

Proposition 3 (Exact publication and per-call credits). *For $W > 0$ and integer $H \geq 1$, the scheduler exhausts its credits and publishes on the H th call, with at most $\lfloor W/H \rfloor$ credits on any call. For $H = \rho N$ with fixed $\rho > 0$, this maximum is $O(\log N)$.*

```

// At a frame boundary: latch settings; compute a = W/H, r = W%H.
// Each active sample call:
capture_input();
if (phase == 0) { latch_input_endpoint(); e = 0; begin_frame(); }
size_t quota = a;
e += r;
if (e >= H) { e -= H; ++quota; }
for (size_t i = 0; i < quota; ++i) execute_next_unit();
if (++phase == H) {
    publish_owned_output(); phase = 0;
    if (window_was_dirty) use_clean_frame_quota();
}

```

Listing 2: Balanced one-hop dispatch. `execute_next_unit` retains stage and traversal state; output stays private until publication.

Proof. After s calls the accumulator rule has issued $sa + \lfloor sr/H \rfloor = \lfloor sW/H \rfloor$ credits. At $s = H$ this is W ; at $s = H - 1$ it is strictly smaller. Each difference is either a or $a + 1$. Substitution of Equation (3) gives the asymptotic bound. $\square$

For a clean $N = 2048, H = 1024, o = 1$ frame, $W = 8194$ and the maximum quota is nine: 1022 calls consume eight credits and two consume nine. With a dirty window, $W = 11266$: 1022 calls receive eleven credits and two receive twelve. At most $\lceil W/H \rceil / 4$ preparation pairs execute in a dirty call, whereas subsequent stages may execute up to $\lceil W/H \rceil$ operations. For Fourier ($o = 2$), the corresponding counts are $W = 9219$ for a clean cache and $W = 12291$ for a dirty cache. Output calls execute at most $\lceil W/H \rceil / o$ bins, limiting the additional concentration of costly coordinate mapping during a window rebuild. Reweighting redistributes bursts between stages; it does not reduce the transform arithmetic. Dense dispatch skips credit over a whole segment, and SIMD butterflies share traversal setup within the same quota. The bound includes window/cache rebuilds, but excludes construction and host lifecycle callbacks. Constant input/control work and frame configuration remain, including an $O(\log N)$ plan-index calculation. No $O(N)$ boundary pass remains in steady-state engine analysis. This count assumes fixed-width scalar/SIMD arithmetic and the bounded module output callbacks; the generic API cannot constrain an arbitrary callback. It does not assert equal duration across credits or bound OS interruptions.

The current implementation also omits multiplication by the exactly unity first twiddle in dense SIMD butterfly segments. This reduces arithmetic without changing scheduling credit or the dependency order. It is a standard FFT simplification, not a new transform algorithm.

D.2 Retained-input proof and plan lifecycle

Incremental packing cannot read a ring that has already overwritten its selected samples. With prepared maxima $N_{\max}, H_{\max}$, the production ring has capacity $C = N_{\max} + H_{\max}$. At selection, the oldest needed sample is $N - 1$ samples behind the newest. At most $H - 1$ new captures occur before completion. Its maximum lag is therefore $N + H - 2 < C$, so it cannot be overwritten before its packing unit. This conservative bound remains valid when capture pauses. The ring and all transform/output arrays are preallocated; no whole-frame capture copy is required.

A maximum-size bit-reversal table serves shorter transforms by right shifts; a maximum-size scalar twiddle table serves them by strides (Appendix E). SIMD lanes share those scalar twiddles. Window weights and smoothing endpoints rebuild only in the scheduled units that use them. Length changes logically clear input and EMA history. Reset cancels partial work and marks partially

rebuilt caches invalid. Increasing the supported hop during a sample-rate callback can grow the ring and allocate; this is outside the allocation-free sample path.

D.3 Smoothing and output ownership

When frequency smoothing is enabled, for magnitudes $a_k = |X[k]|$, reconstruction builds $P_0 = 0$ and $P_{k+1} = P_k + a_k$. Inclusive smoothing bounds $[\ell_k, h_k]$ then give

$$v_k = \frac{P_{h_k+1} - P_{\ell_k}}{h_k - \ell_k + 1}, \quad y_r[k] = \alpha y_{r-1}[k] + (1 - \alpha)v_k. \quad (21)$$

The octave interval is defined in Appendix C: endpoints are floored to bins; if its upper frequency exceeds Nyquist, it is set to Nyquist and the lower endpoint is set to that upper frequency divided by 2^β . With frequency smoothing off, $v_k = a_k$ and prefix/band work is skipped. With $\alpha = 0$, the implementation stores $y_r[k] = v_k$ directly, retaining that history for later averaging. When enabled, the full prefix stage precedes output because a low bin's interval may require higher bins. These are magnitude means, not power means or constant-Q analysis.

For Average control $T > 0$ seconds, the retained convention is $\alpha = \exp(-10\Delta t/T)$; $T = 0$ disables averaging. Spectre uses $\Delta t = H/f_s$. The Fourier module uses its Hop control in seconds and truncates $\Delta t f_s$ to H , retaining a sub-sample quantization difference between the control and actual interval. The previous L/H completion bias is removed. Window coherent-gain factors and module-specific display scaling are preserved; this scheduling change does not redefine amplitude units.

Fourier additionally caches the frequency coordinate and slope gain for each bin in engine-owned storage. A geometry change invalidates a prefix counter in constant time; each scheduled output computes at most one missing entry. Only consecutively initialized entries advance that counter, so cancellation cannot expose stale entries. Magnitude scaling still evaluates every current value. This uses approximately 64 KiB at $N_{\max} = 16384$, with unchanged frame timing and no approximation of the coordinate formulas. Reuse reduces steady-state work; changing geometry every frame removes that reuse. These engineering changes postdate the report's historical timing campaigns and must be measured separately rather than attributed to those data.

Each output unit maps a four-channel bin into the Fourier module's curve points, or writes one Spectre column magnitude. Only after all units finish does the engine publish the destination. A single-producer/single-consumer triple-slot mailbox uses acquire/release atomic exchange to transfer ownership of a completed slot and obtain a reusable slot. The consumer holds its slot until its next successful exchange; the producer never overwrites that slot. A slow consumer may miss intermediate snapshots. The Fourier module has one curve mailbox; the Spectre module has one mailbox per history column. Spectre's full image history and recoloring belong to the UI, outside the engine schedule.

Freeze semantics remain distinct: the Fourier module stops capture but continues analysis of retained input, whereas Spectre pauses both acquisition and the schedule and resumes its partial hop. Continuous-input timestamp equations do not apply across such pauses. Host sample-rate changes cancel partial analysis; Spectre reset still clears and publishes its history columns. These lifecycle costs, UI work, other shared controls, and host scheduling prevent a broad claim of hard real-time safety.

E Implementation Reference

This appendix collects supporting transform and table-construction details formerly repeated in the module manuals. The signal model and one-hop contract remain in the main text; Appendices A and D preserve the transform derivations and detailed scheduling argument. The user manuals cover controls, display interpretation, and practical setting choices.

E.1 Bit reversal and table reuse

For a radix-2 complex transform of length $Q = 2^p$, write an index as $i = \sum_{b=0}^{p-1} i_b 2^b$, where $i_b \in \{0, 1\}$. Its bit-reversed index is

$$R_p(i) = \sum_{b=0}^{p-1} i_b 2^{p-1-b}. \quad (22)$$

For example, $R_3(3) = 6$: the binary index 011 becomes 110. Placing the input at these addresses prepares the iterative decimation-in-time traversal of Equation (12). Listing 3 constructs the table during initialization, not on each sample call.

The production real analyzer prepares a table for $Q_{\max} = N_{\max}/2 = 2^{p_{\max}}$. A frame of real length N needs a complex transform of length $M = N/2 = 2^p$. For $i < M$, its table entry is

$$R_p(i) = R_{p_{\max}}(i) \gg (p_{\max} - p). \quad (23)$$

The leading zero bits of the shorter index become trailing zeros after reversal; the shift removes precisely those zeros. Each packing unit writes its complex pair directly to that address. This fuses permutation with packing and window application instead of requiring a separate array pass.

E.2 Twiddle lookup and batch reference

Let $T[j] = \exp(-i2\pi j/N_{\max})$ denote the maximum-size twiddle table. At a butterfly stage of span s and pair index a , the required factor is obtained by

$$W_s^a = T[aN_{\max}/s]. \quad (24)$$

All supported spans divide $N_{\max}$, so the index is integral. Real-spectrum reconstruction similarly uses $W_N^k = T[kN_{\max}/N]$. Thus changing the transform length does not require rebuilding trigonometric tables. Scalar float or double precision is used for the corresponding scalar instantiation; SIMD channels share the float table.

Listing 4 gives the complete dependency order for a windowed real transform. It is explanatory batch pseudocode, not a function invoked in one production sample call. In the production scheduler the pair, butterfly, and reconstruction loops are suspended between work quotas; window-cache

```
for (size_t i = 0; i < Q; ++i) {
    size_t x = i, reversed = 0;
    for (size_t bit = 0; bit < p; ++bit) {
        reversed = (reversed << 1) | (x & 1);
        x >>= 1;
    }
    reversal[i] = reversed;
}
```

Listing 3: Reference construction of a bit-reversal table for $Q = 2^p$.

preparation is included in the pair units. The output and smoothing stage follows as described in Section 5.

The omitted negative-frequency bins follow from $X[N - k] = \overline{X[k]}$ for $1 \leq k < M$; neither module needs to materialize them. The generic full-spectrum real-FFT class discussed earlier does so. The positive-spectrum convention includes both DC and Nyquist.

E.3 Window and smoothing conventions

Windowing multiplies each selected input sample by its window weight before packing. Tapered windows trade a wider main lobe for different sidelobe behavior [4]; denser FFT bins alone do not remove that tradeoff. Production weights use the existing tabulated coherent-gain correction, $\tilde{w}[n] = w[n]/G_{\text{tab}}$, with the finite-length caveat in Section 3. A frame selects its input endpoint and settings once, so later parameter changes cannot mix window functions within that frame.

For completeness, the inclusive fractional-octave interval used by the prefix-sum smoother can be written explicitly. Let $\Delta f = f_s/N$, $f_k = k\Delta f$, and $\beta > 0$. Define

$$f_h = \min(f_k 2^{\beta/2}, f_s/2), \quad (25)$$

$$f_\ell = \begin{cases} f_k 2^{-\beta/2}, & f_k 2^{\beta/2} \leq f_s/2, \\ f_h 2^{-\beta}, & f_k 2^{\beta/2} > f_s/2, \end{cases} \quad (26)$$

$$\ell_k = \lfloor f_\ell / \Delta f \rfloor, \quad h_k = \min(N/2, \lfloor f_h / \Delta f \rfloor). \quad (27)$$

The upper-edge adjustment preserves the nominal frequency ratio before rounding to bins. The mean includes both endpoints; this is why the prefix formula in Appendix D uses P_{h_k+1} and divides by $h_k - \ell_k + 1$. With smoothing disabled, the magnitude passes through without this range average. DC has the one-bin interval $[0, 0]$.

F Original FFT Experimental Method

The artifact contains a standalone C++11 driver and a Python standard-library runner. The runner records source digests before compilation and rejects a run if those files change during evaluation. No Rack headers, devices, or external FFT libraries are required. The source is compiled using Apple Clang 21.0.0 (`clang-2100.1.1.101`), with `-std=c++11 -O3 -DNDEBUG -Wall -Wextra -pedantic`; fast-math is not enabled. The recorded host is an Apple M1 Pro with 16 GiB of memory, running macOS 26.6.2. Measurements were collected on September 28, 2026. No core affinity, real-time thread policy, frequency locking, or exclusive machine reservation was imposed.

```
M = N / 2
for n = 0, ..., M-1:
    z[R[n]] = u[2*n] + i*u[2*n+1] // u is already windowed
for span = 2, 4, ..., M:
    for group = 0, span, ..., M-span:
        for pair = 0, ..., span/2-1:
            execute_butterfly(z, span, group, pair)
for k = 0, ..., M:
    X[k] = reconstruct_positive_bin(z, k)
return X[0], ..., X[M]
```

Listing 4: Batch reference for the packed real transform. R is the bit-reversal table for M; butterfly updates use Equation (12), and reconstruction uses Equation (14) and its endpoint rules.

F.1 Numerical and schedule verification

Lengths are all powers of two from 4 through 4096, using both `float` and `double`. Fixtures are silence, an impulse, DC, a Nyquist alternating sequence, a bin-centered sinusoidal mixture, and deterministic pseudorandom samples. Each is tested with a rectangular window and a periodic Hann window. The pseudorandom generator is the explicitly specified 32-bit recurrence in the driver; it is not dependent on a standard library’s distribution implementation.

For every fixture/window/precision/length combination, `compute()` is compared bitwise against repeated `step(H)` for $H \in \{1, N/4, N/2, N, N+7\}$. The list intentionally retains coincident horizons at the smallest size. Every completion count is checked against $\lceil B_r / \lceil B_r / H \rceil \rceil$. These checks examine the actual header, not a rewritten FFT model.

For lengths through 128, a separately written $O(N^2)$ complex DFT checks every output bin, including DC, Nyquist, and the conjugate half. The reference uses the actual windowed input values presented to the FFT and reports

$$E = \frac{\max_k |\hat{X}[k] - X_{\text{ref}}[k]|}{\max(1, \sum_m |u[m]|)}. \quad (28)$$

The denominator avoids unstable relative error at spectral nulls. The acceptance threshold is $64\epsilon_T$ for the tested scalar type; it is a regression threshold for this small-size reference campaign, not a proven error bound for all lengths. Although the reference uses `long double`, that type has 53 significand bits on this host. In particular, the double-precision reference is an independent formulation, not a higher-precision oracle.

F.2 Timing comparison

The timing workload uses one scalar `float` real transform, lengths $N \in \{1024, 2048, 4096, 16384\}$, and horizons $H = N/4$ and $H = N/2$. A fixed pseudorandom frame is multiplied by an explicitly constructed periodic Hann window. Transform construction, table initialization, window construction, and fixture generation are excluded. Packing allocation, window multiplication, copying, bit reversal, and reconstruction are included.

Two modes share the same FFT implementation. The complete mode performs `buffer()` followed by `compute()` in the first simulated sample call. The incremental mode performs `buffer()` in that call and invokes `step(H)` on all H calls, retaining any early-completed result until the next fixed frame boundary. This controlled cadence holds the number of frames constant; it deliberately differs from the original modules’ immediate-restart behavior so that a faster update rate does not confound the scheduling comparison.

Each mode receives 16 warmup transforms and two measured passes of 64 frames. The first pass times whole frames with no clock reads inside the incremental loop; the complete baseline has no FFT work in the other $H - 1$ logical calls. The second pass times every simulated call, records preparation and completing calls separately, and retains per-run p99 and maximum durations. Output magnitudes are consumed through a volatile checksum outside the timed regions to prevent dead-code elimination. Both modes compute full spectra.

There are nine paired repetitions per size/horizon combination. Execution order alternates between complete-first and incremental-first on successive repetitions. Results include medians and full ranges across repetitions. For paired ratios, the runner resamples the nine pairs 10,000 times with a fixed seed and reports percentile bootstrap intervals. These intervals describe within-session repeat variability; they do not account for different hardware, operating systems, compilers, or background loads. They also do not quantify sensitivity to executable or memory layout. Mytkowicz

et al. show that layout changes induced by link order or environment size can bias performance comparisons [28]. Alternating the two modes’ execution order is not equivalent to randomizing those experimental conditions, so a narrow within-session interval does not establish robustness to them.

The timer is `std::chrono::steady_clock`. An empty start/end pair has median 0 ns and p99 42 ns in the recorded calibration. Tiny per-call observations are therefore quantized and instrumentation-sensitive. Clock overhead is not subtracted. The separate frame pass avoids multiplying those clock reads by H when estimating aggregate work.

G Original FFT Results

G.1 Numerical agreement and completion

All 1320 incremental-versus-complete schedule cases passed, with bitwise-identical completed spectra within each tested type. The independent reference checked 6048 complex bins across both precisions. The maximum scaled errors were 8.69×10^{-8} for float (rounded upward) and 1.06×10^{-14} for double (rounded upward). These measurements support correctness for the tested fixtures, sizes, and windows; they do not replace an error analysis of every supported type or the module’s amplitude calibration.

The schedule tests confirm Table 10 and the distinction between “no later than H ” and “exactly at H .” Increasing the requested horizon can leave the actual completion interval unchanged until the integer quota changes. Thus frame rate is a staircase function of the horizon under completion-driven restarts, not generally its reciprocal.

G.2 Peak work placement

Table 11 reports the $H = N/2$ subset; the artifact also contains $H = N/4$. Figure 3 includes the full range of observed maxima across repetitions. The median of run maxima is much smaller in incremental mode, but individual maxima vary considerably. At $N = 2048$, for example, one incremental run reaches 21.209 microseconds even though its median run maximum is 1.625 microseconds. Such observations are consistent with an ordinary scheduled process and are retained rather than removed as outliers.

The incremental preparation-call medians are 0.542, 1.125, 2.709, and 10.667 microseconds for the four lengths. Completing-call medians are 0.750, 1.500, 3.042, and 12.375 microseconds. Their growth is consistent with the bulk work identified by source analysis, although timings alone do not prove an asymptotic bound. Distributing only the butterfly traversal leaves both ends of the

N	Frame time (μ s)		Run maximum (μ s)		Peak ratio Paired 95% interval
	Complete	Incremental	Complete	Incremental	
1,024	7.619	7.518	7.917	0.875	[0.083, 0.508]
2,048	16.662	15.915	17.291	1.625	[0.091, 0.111]
4,096	36.620	35.603	39.458	3.333	[0.079, 0.296]
16,384	169.510	163.844	179.625	13.125	[0.073, 0.113]

Table 11: Medians across nine repetitions, $H = N/2$. Frame time comes from the aggregate pass; run maximum comes from the separately instrumented pass. Intervals are bootstrap intervals for the median paired incremental/complete run-maximum ratio, not ratios of the displayed marginal medians and not worst-case bounds.

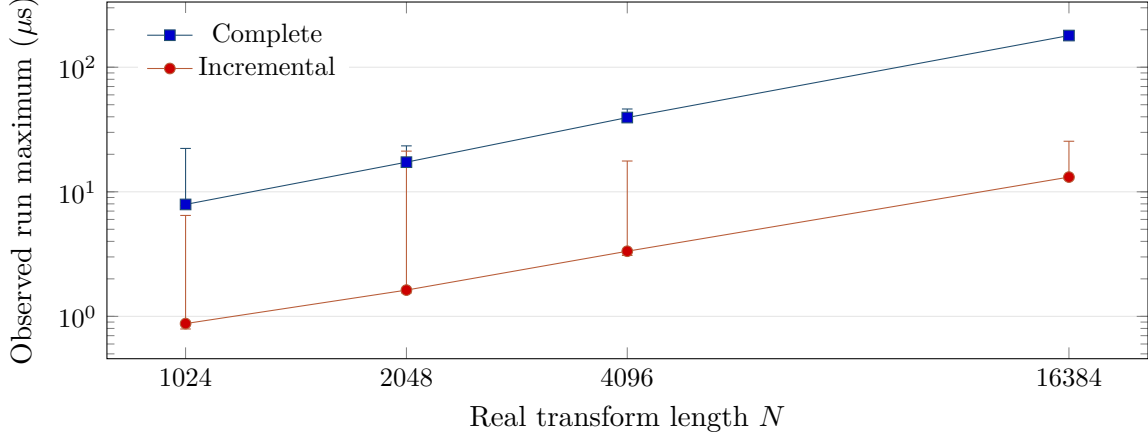


Figure 3: Median and full min-max range of per-run maximum call duration over nine repetitions, $H = N/2$. Vertical bars are observed ranges, not confidence intervals. Both axes are logarithmic; horizontal ticks identify the actual transform lengths. Measurements include RFFT preparation and reconstruction, but exclude module smoothing and graphics preparation.

transform visible in the cost profile.

A striking measurement pitfall is that the median per-run p99 is approximately 42 ns for *both* modes at all four lengths in this subset. The complete mode’s expensive call occurs only once per H calls, below the top one percent for these horizons. A p99-only comparison would largely miss the burst being studied. Phase-specific measurements and run maxima are needed alongside quantiles; none of them by itself is a proven worst-case execution time.

G.3 Aggregate work and interpretation

The median paired incremental/complete frame-time ratios for increasing N are approximately 0.979, 0.955, 0.972, and 0.967. The corresponding percentile intervals are [0.973, 1.005], [0.955, 0.956], [0.960, 0.977], and [0.963, 0.968]. These modest differences are specific to the two loop structures as optimized by this compiler on this workload. They do not indicate a reduction in the butterfly count, nor establish an algorithmic throughput advantage. A different compiler, batching policy, cache state, or vectorized library could reverse the result. The robust interpretation of this campaign is the change in work placement within this implementation, with broadly comparable total frame cost.

This original campaign uses repeated fixed inputs, warm tables, and a scalar standalone process. They omit ring insertion, Rack parameter access, SIMD channels, frequency/time smoothing, display conversion, GUI consumption, and other modules. They therefore cannot establish whole-patch dropout rates or a maximum number of safe analyzer instances. Neither timing campaign measures a running Rack session.

H Complete-Pipeline Prototype Evaluation

A second campaign informed the production choice. Its exact sources and raw summaries are archived separately in `docs/whitepaper/data/pipeline/`. The prototypes compare: (i) the original arithmetic with controlled exact cadence (`legacy`); (ii) candidate arithmetic executed in boundary bursts across four stage slots (`stage_burst`); (iii) a serial balanced sequence within one hop (`serial`);

and (iv) four overlapped stages across four hops (**pipeline**). Modes (ii)–(iv) share positive-bin specialization and prepared smoothing bounds. The stage-burst and overlapped modes share storage and four-hop latency and therefore isolate placement most closely. Serial also changes stage storage and latency; its comparison with stage bursts is informative but cannot isolate dispatch cost alone.

The host and compiler are the Apple M1 Pro and Apple Clang 21.0.0 described above. All modes use C++11 with `-O3 -DNDEBUG -Wall -Wextra -pedantic`, scalar float, deterministic noise, normalized periodic Hann, $\alpha = 0.8$, and 48 kHz. Sizes are $N \in \{1024, 4096, 16384\}$, hops $H = N/4, N/2$, and smoothing off or one-third octave. Twelve repetitions rotate the mode order; each pass has 20 warmup and 64 measured hops. Separate passes measure total hop time, individual samples, and synchronous 16-, 64-, and 256-sample blocks. These are simulated blocks, not device callbacks. Empty timer median/p99 is 41/42 ns, with no subtraction. Historical phase timings separately confirm that packing, reconstruction, and postprocessing remain concentrated.

Table 12 shows an approximately 80% reduction in the median observed block maximum for serial versus legacy in this workload. Serial’s 51.392 microseconds per hop exceeds the stage-burst control’s 50.327. The fully matched overlap comparison also costs more, at 55.145 microseconds: scheduling is not free. Across all 12 size/hop/smoothing settings, median paired serial/legacy total-time ratios span 0.920–1.212; overlap/legacy spans 0.986–1.585. Peak reduction can coexist with greater total CPU cost. Four-hop overlap adds $3H$ samples of age: 64 ms here and 256 ms for $N = 16384, H = 4096$. It did not earn that extra latency and storage in this campaign.

The measured prototypes are not the integrated production implementation. Their arithmetic, cache policy, and adapters informed the successor, which subsequently gained maximum-size plans, live cache rebuilding, SIMD support, and module-owned output. Do not use these timings as a production speedup. The campaign retains all per-repetition summaries, including preemption-sensitive maxima, but no individual call traces. It remains a single-host study without Rack graphics, actual audio callbacks, or an optimized-library baseline.

Production correctness is checked separately: 1,934,764 standalone assertions in seven cases cover spectra, cadence, retained-input wraparound, all 15 window functions, cache invalidation, reset, and allocation-free live changes. A separate Rack-linked suite includes scalar/SIMD agreement and coordinate mapping (3,807,481 assertions in four cases); module lifecycle and serialization suites also pass. The standalone suite passes AddressSanitizer and UndefinedBehaviorSanitizer. A Rack plugin build and native display inspection with context recreation pass, but no full interactive Rack/DAW workload or integrated production timing campaign was performed. These checks support the implementation contract, not a device deadline guarantee.

I Extended Related Work

This appendix preserves the detailed comparisons supporting the positioning in Section 2.

Mode	Hop (μ s)	Block max (μ s)	Max range (μ s)	Age (samples)
Legacy	56.009	24.625	24.291–36.292	1023
Stage bursts	50.327	45.751	43.125–57.625	4095
Serial one hop	51.392	4.875	4.792–9.167	1023
Four-hop overlap	55.145	5.229	4.000–18.917	4095

Table 12: Selected prototype workload: $N = 4096, H = 1024$, one-third-octave smoothing, 64-sample blocks. Hop time and block maximum are medians of 12 run summaries from separate passes; range is the full range of those maxima. Age is measured from the frame’s newest input.

Cooley and Tukey’s factorization is the mathematical foundation of the complex transform used here [1]. Real-input symmetry and real FFT algorithms have a substantial independent literature, including Sorensen et al. [2]. Short-time Fourier analysis and synthesis provide the frame-based interpretation [3]; window selection and its spectral consequences are treated systematically by Harris [4]. The packing and windowing used in the plugin are applications of this established body of work.

Time-distributed FFT computation also predates this implementation. Lo and Lee construct split-radix butterfly work as data become available during acquisition [5, 6]. Their moving FFT additionally combines real-time scheduling with recursive reuse across shifted frames [7]. These methods are closer to acquisition-aware computation than the present implementation, which selects a complete window before processing it. The successor retains that window in a larger ring while packing it incrementally.

Convolution research directly addresses the distinction between aggregate cost and processing bursts. Borß distributes long-partition work across processing cycles, allowing less spectral-product work in cycles containing transforms, and staggers four filter schedules to smooth their combined load [8]. This provides an earlier plugin-level precedent for load distribution, including the importance of coincident processing bursts. Hurchalla distributes FFT processor and memory load across incoming input blocks in one thread for low-latency convolution [9]. That input–output latency objective differs from our selected window’s newest-sample-to-publication delay.

Battenberg and Avizienis compare cooperative and preemptive implementations of partitioned convolution [10]. They use decimation-in-frequency decomposition to distribute forward transforms, spectral multiplication, and inverse transforms across callbacks. Thus, scheduling operations beyond the FFT itself also has direct precedent. Their preemptive implementation performs best in their evaluated scenarios; the cooperative implementation fits their host-plugin execution model but requires more implementation effort. This motivates a worker-based comparison without establishing a performance ranking for this analyzer. The present study specifies a windowed spectral-analysis sequence, an exact-hop quota, retained-input capacity, cancellation, and output ownership. Its complete chain experiments use single-block overlap-save with identity and a short FIR as correctness fixtures; they do not evaluate the long-filter partitioning problem addressed by these convolution systems.

Wefers places these techniques in a broader treatment of real-time partitioned convolution, including manual preemption of processing chains and FFT subdivision [11]. This motivates evaluating the granularity of resumable work rather than assuming butterfly-level suspension is optimal.

Liu, Yan, and Zou further study time-distributed FFT/IFFT algorithms for online control on general-purpose processors [12]. The existence of these approaches rules out a broad novelty claim for distributing transform work or a processing chain in time.

Průša and Holighaus provide an application-level comparison in their real-time spectrogram visualizer using reassigned non-uniform filter banks [13]. Their Section 4.2 places incoming audio in a lock-free circular buffer and performs analysis on a separate thread polling at 60 Hz. They distinguish overlap, computation, and repaint delay, and identify polling as a source of jitter. This is a concrete alternative execution model for spectral visualization. Its filter-bank representation differs from our fixed-window FFT; the relevant comparison here is the placement of analysis work and the path from captured samples to visible output.

A separate family of methods reuses work between overlapping windows. The sliding DFT updates spectral bins recursively as samples enter and leave a window [14]. Lyons and Howard present efficient, guaranteed-stable sliding DFT networks, including a variant with a freely selectable analysis frequency [15]. Recursive analysis therefore includes designs that explicitly address stability;

recursion alone is not grounds to reject it. Tree-based sliding-window transforms reuse intermediate calculations [16]. The plugin’s resumable FFT does not reuse work across frames: each new frame begins a fresh transform.

Garrido’s feedforward STFT reuses radix-2 decimation-in-time butterflies across successive one-sample window shifts [17]. Its feedforward graph avoids the feedback accumulation of error in the iterative formulation it compares, while retaining intermediate results in buffers. This is distinct from claiming zero numerical error. The paper also exploits real-input symmetry. Its basic formulation uses a rectangular window and a one-sample hop; matching our window and output cadence requires additional consideration. Neither its hardware comparisons nor its small MATLAB timing example establish a performance ranking for this plugin.

Park and Ko’s hopping DFT updates a spectrum from the spectrum one hop earlier, without computing every intervening sample’s spectrum [18]. This makes overlap reuse relevant to multi-sample hops as well as sample-by-sample analysis. It changes the arithmetic across frames, whereas our quota changes the placement of work within a frame. Lower aggregate work alone would not establish a smaller processing burst.

Windowing must also be matched in a comparison. Rafii derives frequency-domain kernels for windowed sliding transforms [19]. The Hann and Blackman examples have three- and five-point kernels, respectively; sparsifying more general kernels can introduce approximation. Efficient windowing therefore need not rule out overlap reuse, but its cost and accuracy depend on the window. These kernels do not by themselves establish the numerical stability of the underlying recurrence. Selecting among these methods requires matching bin count, hop, window convention, normalization, and numerical behavior. Our experiments do not measure that tradeoff against sliding or hopping methods.

Eleftheriadis, Garrido, and Karakonstantis combine partial window overlap with frequency-domain Hann windowing [20]. Their frequency decomposition updates a prior spectrum using corrections from entering and leaving sample blocks. This connects multi-sample hops and efficient windowing in one design. The reported fixed-point ASIC results motivate a software comparison, but do not establish CPU savings or long-run floating-point accuracy. Scheduling the correction work within our hop budget would be a further adaptation, not a result demonstrated by that paper.

Within a single transform, Bécoulet and Verguet give a depth-first iterative conjugate-pair FFT [21]. Their traversal uses integer indexing without recursion or precomputed index tables, with constant auxiliary space for traversal and input reordering; this is not a constant-space claim for all transform storage. The approach offers another way to organize locality and traversal state. It does not itself provide our resumable scheduling contract: bounded suspension units and their cost would need to be established separately.

Optimized libraries such as FFTW use planning, generated kernels, and hardware-sensitive decompositions to improve actual execution time [22]. The present evaluation separates two questions. Matched first-party batch/resumable controls hold arithmetic substantially fixed to study work placement. External-library adapters measure the practical cost of using a different transform kernel, including the layout conversion and output processing required at each declared workload boundary. FFTW uses fresh, single-threaded `FFTW_MEASURE` plans without imported wisdom; plan identities are retained because planning is part of the experimental condition.

PFFFT is a particularly relevant practical alternative: Rack exposes its real transform through `dsp::RealFFT` [23]. The wrapper offers ordered and unordered output, so a spectral-analysis comparison must include the work needed to obtain the required bins. The campaign records the linked Rack/PFFFT source and binary identities and vector path; different forks and builds are not assumed interchangeable. A further control retains an opaque PFFFT call inside incrementally scheduled preparation and output work, testing how much burst reduction requires suspending

the transform itself. Apple Accelerate’s vDSP provides a platform-specific baseline for the Apple Silicon host [24]. Its reusable setup, packed real-transform layout, and scaling conventions are included in the adapter contract. Both libraries are measured in the replacement campaign, with numerical auditing and provenance retained. Their citations document the software interfaces, while all comparative timings in this report come from the accompanying experiment. Table 13 summarizes the design distinctions without imposing a universal performance ranking.

Approach	What changes incrementally	Principal distinction
Batch FFT	A complete frame per invocation	Implementation can optimize the whole call.
Resumable FFT here	Traversal of one frozen frame	Caller controls work placement; no inter-frame reuse.
Acquisition-aware FFT	Available subtransforms during acquisition	Depends on input-arrival order and transform graph.
Sliding/hopping/tree/feedforward methods	Spectra or intermediate results across windows	Reuses overlap; different state and error analysis.
Worker-thread FFT	Execution on another scheduled thread	Requires ownership, synchronization, and deadline policy.

Table 13: Conceptual comparison. These are design distinctions, not a performance ranking.

J Further Evaluation

The confirmation study supplies optimized-library baselines, repeated sessions, inverse jobs, filtering chains, and selected state/load extensions. Its main remaining questions concern different execution models, stronger replication, and actual host integration, rather than another measurement of the same grid.

Asynchronous and overlap-reusing alternatives. A worker can remove analysis from the engine thread when asynchronous delivery is acceptable, as considered by Battenberg and Avizienis [10]. A matched experiment should include capture, queueing, missed publications, consumer contention, and actual spectrum age. For heavily overlapping windows or a small requested bin set, hopping, windowed sliding, and feedforward methods deserve separate comparisons [17, 18, 19, 20]. Equivalent windowing, normalization, smoothing, and output processing are essential. A periodic Hann window permits an exact short window kernel; approximate truncated kernels would require a separately stated accuracy target. None of the present FFT backend results ranks these alternatives.

Numerical stress and applications. Van der Byl and Inggs distinguish stability from useful numerical accuracy in sliding structures [25]. Long streams with weak tones beside strong ones, silence after a strong signal, and phase-sensitive processing would extend the current deterministic fixtures. The binary64 references here are not exact arithmetic. Non-power-of-two lengths, more windows, and application-specific error budgets remain untested. The overlap-save identity and three-tap FIR establish processing semantics; realistic long impulse responses and frequency-dependent effects would be needed before recommending FFT filtering for an application. A complete bounded-stage inverse pipeline would also have to distribute the bulk initialization identified in Section 6.

Device and display behavior. For a visualizer, producer publication is only one component of perceived latency. The decomposition of Průša and Holighaus [13] motivates measuring consumption and repaint separately. The mailbox permits superseded snapshots when the display is slower than production. A Rack study should measure the real callback schedule, driver-reported underruns, concurrent displays, and patch load. Callback phase and analyzer count matter because individually tolerable bursts can coincide, as staggered convolution schedules also illustrate [8].

Skare separates continuous throughput from periodic callback measurements and emphasizes their difference from plugin invocation within a DAW [26]. Balasubramaniam et al. distinguish plugin timing, session callback utilization, driver underruns, and worker inference underruns under controlled contention [27]. Their virtual driver does not include physical-interface behavior. These methods inform the next host experiment; their GPU and inference results are not FFT-backend evidence. The present synchronous harness measures computation, not audio reliability.

Replication and granularity. The three temporal blocks provide observed variation on one host. Additional days, builds, architectures, compilers, and operating systems should be chosen before making general platform claims. Kalibera and Jones provide a framework for allocating repetitions across sources of variation [29]; layout and environment bias also require attention [28]. Longer sessions would help characterize infrequent stalls but would still not supply a safe worst-case bound [30].

The current weights distribute operation density without making operations equal in duration. Following Wefers’ discussion of transform subdivision [11], a controlled study could vary butterfly batches or small subtransforms under the same hop and publication constraint. The depth-first traversal of Bécoulet and Verguet [21] suggests comparing immediate and resumable forms of the same alternative kernel across working sets and analyzer counts. That would separate locality and factorization from suspension. Hardware-calibrated weights might improve typical behavior, but would add assumptions rather than prove execution time on a general-purpose OS.

K Reproducibility and Source Map

The code and report sources are available in the [Fourier repository](#). The report’s modular sources live in `docs/whitepaper/`; the submission exporter expands them into a standalone TeX file. Typesetting performs no experiments, dependency fetches, or network requests. From the repository root:

```
python3 docs/whitepaper/tools/comparison_paper.py --check
make -C docs/whitepaper check
make -C docs/whitepaper arxiv
```

The outputs are `paper.pdf` and `fourier-arxiv-source.tar.gz` in `docs/whitepaper/.build/`. The latter is a portable source archive. After extraction into an empty directory, compile with

```
latexmk -pdf -pdflatex='pdflatex -no-shell-escape %0 %S' fourier.tex
```

These commands validate and reproduce document assets; they do not repeat the timing experiment.

Confirmation evidence

`docs/whitepaper/data/comparison-012/` contains the frozen designs, host records, compressed campaign metadata, all summary/process tables, numerical coverage, and the measured first-party source. The nominal revision is `12ac3236e332b7d0a01b2d750fa07a0849132341`, with the recorded

runner fix excluding Finder metadata from source inventories. The exact measured source archive has SHA-256

`bd46acd4281fdb6099908dab857351270554d9b93563620356bd3b8833000724.`

The source, dependency, executable, configuration, and plan identities in each campaign determine what was measured; the later manuscript revision does not. Compiler commands and numerical policies are preserved with those identities.

The publication generator reads the immutable committed CSV tables, verifies their source-manifest hashes, and writes selected LaTeX tables, vector plot coordinates, numerical macros, and a provenance receipt. Its check mode rederives those outputs without modifying measurements. Selected workloads have complete configuration identities; length trends keep all three primary lengths, and the full grid remains available even where it is not typeset. Historical results are not merged into these selections.

Individual callback observations, full reports, and portable audit tooling are preserved in six *local, untracked* bundles identified by `data/comparison-012/bundles.json`. They have not been deposited at a public archive. A fresh repository clone can check the published tables and regenerate the selected document assets, but cannot rederive all statistics from individual observations without those bundles. The local bundle README gives unpacking and rederivation commands. All five confirmation bundles passed table rederivation, and both pilot reviews reproduced from their bundle. Native executables, Rack SDK bytes, and dependency archives are explicitly omitted from the portable bundles; their measured hashes remain recorded. Repeating timings requires separately acquiring the recorded dependencies and preparing a measurement host. Public deposition of the raw bundles is a remaining dissemination step, not a completed result of this report.

Historical campaigns and implementation

The original campaign’s exact inputs are in `docs/whitepaper/data/source.tar.gz`; its README gives isolated extraction and reproduction commands. The archive restores historical math-header paths and the driver hashes in its own metadata. The complete-pipeline prototype sources and dependency headers are in `docs/whitepaper/data/pipeline/source.tar.gz`. That campaign retains 2880 per-repetition timing summaries and 27 baseline phase observations; it did not retain individual timed calls. Its README and metadata describe those distinct methods. Neither dataset measures the current production successor.

The production paths are `src/dsp/spectrum_analysis.hpp`, `src/SpectrumAnalyzer.cpp`, and `src/Spectrogram.cpp`. `src/rack_extensions/display_mailbox.hpp` defines snapshot ownership; `src/rack_extensions/spectrum_coordinates.hpp` maps Fourier’s bins. `src/dsp/fft.hpp` supplies the reusable forward and inverse components. The comparison adapters and fixtures live in `benchmark/paper/`; `docs/whitepaper/benchmarks/` owns their campaign/report workflow. These research adapters are not integrated into the Rack modules. Production test and build commands remain in `CONTRIBUTING.md`.

L Data and Software Availability

The source code uses GPL-3.0-or-later, with separate visual-asset terms in the repository license. Compact evidence and measured sources accompany the report; raw observations remain in local bundles (Appendix K). No public data or arXiv deposit is claimed. The repository’s `CITATION.cff` and report’s `CITATION.bib` identify manuscript version 3.